

Distributed Observability for Multi-Agent LLM Systems: A Shared-Log Approach on HPC Clusters

Anna Monsó

July 31, 2026

Abstract

Applications built on large language models (LLMs) are becoming multi-agent and are moving onto clusters, yet to our knowledge no existing observability tool covers both facts at once: LLM-observability platforms capture rich agent semantics but have no notion of physical topology, while HPC and fleet monitoring knows node health at scale but nothing about agents. This thesis fills that gap, starting from one observation: a multi-agent execution is already a causally-ordered event log, so a distributed shared log is its natural storage substrate. Building on ChronoLog, a distributed, tiered log store that places a capture process on every cluster node, we design and implement a drop-in observability system for LLM-agent fleets. Two capture paths feed the log (LLM turns through a spooled capture worker; inter-agent calls through per-node collectors) with at-least-once capture and effectively exactly-once ingest, and a hot/cold read design works around the log’s ingest-optimized acceptance window to serve real-time views from the same event stream. On this substrate we build fleet-scale analyses: failure detection with signature clustering and zero-cost diagnosis, per-node health rollups whose cost is independent of event volume, and cross-host critical-path tracing. We validate the system end-to-end with real Claude agents on live 4- and 8-node ChronoLog clusters (28/28 automated checks, up to 32 concurrent agents) and measure it throughout: capture costs the agent 3.6 ms per turn (statistically invisible against real LLM latency), events reach the operator’s screen in 10.7 ms against a 20–30 s durable drain, fleet queries stay flat as event volume grows 50×, and a ground-truth fault study scores 580/580 detections with zero false positives. In a timed case study, a first-time operator produced a complete, verified cross-layer diagnosis through the system in 72 seconds, a task they could not begin from raw logs.

Acknowledgements

Contents

1	Introduction	8
1.1	Context: Multi-Agent LLM Systems on Clusters	8
1.2	The Gap: Agent Semantics vs. Cluster Topology	9
1.3	Key Insight and Approach	9
1.4	Contributions	10
1.5	Document Structure	10
2	Background and State of the Art	11
2.1	Distributed Systems Fundamentals	11
2.1.1	Time, Ordering, and Causality	11
2.1.2	Delivery Semantics	12
2.2	Distributed Tracing	12
2.2.1	Lineage: From Dapper to Pixie	12
2.2.2	Critical-Path Analysis and Self-Time	13
2.2.3	Sampling and Its Limits at Fleet Scale	13
2.3	The Distributed Shared Log Abstraction	14
2.4	ChronoLog	15
2.4.1	Architecture: Visor, Keepers, Grapher-Player	15
2.4.2	Data Model: Chronicles, Stories, Events	16
2.4.3	Design Point: Ingest-Optimized, Delayed Reads	16
2.5	LLM Agents and Multi-Agent Systems	16
2.5.1	The Agent Loop, Tool Use, and Delegation	16
2.5.2	Observable Signals	17
2.5.3	Failure Modes of LLM Agent Fleets	17
2.6	State of the Art and the Gap	18
2.6.1	LLM and Agent Observability	18
2.6.2	HPC and Fleet Monitoring	18
2.6.3	The Empty Intersection	18
3	Problem Statement and Requirements	20
3.1	Target Scenario	20
3.2	Requirements	21
3.3	Non-Goals	21
4	System Design	23
4.1	Overview: Two Paths, One Substrate	23
4.2	Mapping Agent Semantics onto the Log	23
4.3	Write Path and Delivery Semantics	25
4.4	The Read Problem: Hot and Cold Paths	26
4.5	Process and Failure Isolation	27
4.6	Extensibility: Adapters and Capabilities	27

5	Fleet-Scale Analytics on the Log	29
5.1	Live Topology Coupling: Scenario Graph and Cluster Graph	29
5.2	ChronoDoctor: Detection, Clustering, and Diagnosis	29
5.3	Fleet Health at Scale	30
5.4	Cross-Host Critical-Path Tracing	31
5.5	The Operator Surface: Five Views over One Log	31
6	Evaluation	36
6.1	Methodology	36
6.2	Capture Overhead	37
6.3	Freshness and Query Latency	38
6.4	Detection Quality	39
6.5	Scalability	40
6.6	Functional Validation	41
6.7	Case Study: Fault Injection and Localization	41
6.8	Discussion and Limitations	43
7	Conclusions and Future Work	45
7.1	Conclusions	45
7.2	Future Work	46

List of Figures

2.1	A space-time diagram of three processes exchanging messages m_1 – m_3 . Causality flows along process lines and messages: $a \rightarrow c \rightarrow e \rightarrow d$ is a causal chain, so a <i>happened before</i> d ; b and e are <i>concurrent</i> (neither could have influenced the other), despite b being later on the wall clock.	12
2.2	A trace rendered as a waterfall. Grey spans form the critical path $A \rightarrow C \rightarrow C_2$, the chain that determines end-to-end latency; white spans (B , C_1) overlap it and cannot shorten the request. Each hop is charged its <i>self-time</i> : C ’s 720 ms minus the 400 ms covered by its slowest child leaves 320 ms of C ’s own.	13
2.3	The shared-log abstraction. Producers append at the tail; an event’s position in the log is its permanent place in history. Readers consume independently at their own offsets, imposing no back-pressure on producers or on each other. . .	14
2.4	ChronoLog’s architecture. A keeper on every compute node takes node-local appends into memory (the ingest tier); the visor coordinates clients; the grapher asynchronously drains, merges, and time-orders sealed chunks into a read-optimized archive. The drain imposes an acceptance window during which fresh events are not yet readable (20–30 s measured end-to-end here, with a 180 s window configured at the grapher). This is the property the read path of Chapter 4 is designed around.	15
2.5	(a) The agent loop: each cycle through the LLM is one <i>turn</i> , appending the tool’s observation to the agent’s context. (b) Multi-agent delegation: once agents are tools, agent A calls agent B (possibly on another node), which calls C in turn, each call a start with a matching done (dashed) carrying status and latency. The result is a partially ordered event log with delegations as its cross-process edges.	17
2.6	The observability landscape for multi-agent LLM systems on clusters, along its two capability axes. Camp 1 and Camp 2 each occupy one off-diagonal quadrant; their intersection (agent semantics coupled to physical fleet topology) is unoccupied among the surveyed tools, and is the quadrant this thesis targets.	19
4.1	The two-path architecture. Path-A carries LLM turns through a node-local spool drained by a capture worker; Path-B carries inter-agent calls through a per-node collector. Both persist to ChronoLog through the node-local keeper; the dashboard reads ChronoLog (cold, full fidelity) and a local hot store (live).	24
5.1	The Scenarios view, live: 10 agents across 8 host groups, every inter-agent call an edge (errors red), with live message and error counters. Clicking a node opens the agent’s conversation, the capture-to-conversation join of R1.	33
5.2	The Health view after a Doctor scan over a large demo corpus: 12 ranked incidents from ~ 200 k scanned turns (left), and the selected incident’s diagnosis card (right) carrying root cause, remediation, blast radius, confidence, and evidence samples. <i>Recurring</i> badges are read from the cross-run incident memory.	33

5.3	The Cluster view on a live 4-node deployment: ChronoLog components (visor, grapher, player placement; one keeper card per node with its event and story counts and last-drain age) alongside the live cluster graph and SLURM capacity. The headline freshness stat reports the <i>measured</i> last-drain age (§6.3); the grapher’s configured 180s acceptance window is relegated to its tooltip.	34
5.4	The Fleet view: 256 nodes, one cell each, rolled-up health worst-first. Nine critical nodes are the immediate Q4 answer. Rollup-backed reads keep this view interactive independent of event volume (§6.5).	34
5.5	The Traces view: critical-path waterfall for the slowest trace of a scenario. The header names the bottleneck (the <code>comp-04</code> hop, 12.0s self-time of 21.6s wall-clock); the top bar shows self-time per hop along the critical path.	35
6.1	The freshness and query-latency spread on one log axis (p50, live 8-node deployment). Live push takes single-digit milliseconds and scenario-scoped reads stay sub-second, while population-wide reads that fold the raw archive sit near 2s on this corpus. That is the linear-in-events cost the rollup design of §6.5 flattens to roughly 20 ms.	39
6.2	Fleet-health query latency vs. stored event count (16 nodes, 60-minute window, p50 over 20 requests). Pre-aggregated rollup reads are independent of event volume; the raw-fold fallback is linear in it. Measured offline via <code>bench_rollup_scaling.py</code> in <code>scripts/eval/</code>	41

List of Tables

4.1	The chronicle/story schema: chronicles partition by data kind, stories by entity. Session-scoped stories serve the conversation and memory reads; the per-scenario edge story serves graph and trace reads; the rollup and index stories are written by the capture side so that readers never scan.	25
6.1	The evaluation at a glance. Every number is measured; the sections referenced give conditions, distributions, and caveats.	37
6.2	Capture overhead, measured on a live 8-node deployment (compute nodes, NFS state directory; $n=10\,000$ per latency row). The headline row is the end-to-end A/B turn delta over real LLM turns: its 95% confidence interval spans zero: capture is statistically invisible inside the workload’s own variance. Capture worker CPU is the mean of a bursty profile (idle between 5 s drain passes, peaks of 85% during them); the wire number is amortized over batches of ≤ 64 (658 B unbatched).	37
6.3	Freshness and query latency on the live 8-node deployment (dashboard node, $n=100$ per query row, $n=1\,000$ paced events at 20 ev/s with zero loss for the SSE rows; corpus $\approx 22\text{ k}$ archived events). The hot path sits three orders of magnitude inside the measured drain: an event is on the operator’s screen $\sim 1\,900\times$ sooner than it is readable from the archive (p50 drain over p50 push).	38
6.4	ChronoDoctor detection quality against a ground-truth fault manifest (offline, seeded through the production write paths; <code>e6_doctor_quality.py</code> in <code>scripts/eval/</code> , seed-deterministic; results identical at $n=25$ and under a different seed). “Cards” is the number of incident cards produced per fault kind; “Diagnosis” marks whether the heuristic diagnosis names the planted root-cause category, with the engine’s stated confidence; “Recurring” marks whether the second scan badged the incident with its prior count.	40
6.5	Timed diagnosis of the injected fault, $n=1$ operator (first-ever exposure to the task). Dashboard trials repeat the same questions, so passes 2–3 measure a practiced operator. The raw-log attempt ran <i>after</i> the dashboard trials (answer leakage, if any, favored the raw-log path) and still produced no answers: the operator could not establish a starting point in the JSONL directories. The supplementary row is a fresh LLM agent given only the three questions and the data directories (no ground truth, shell text tools only); machine wall-clock is not comparable to human minutes and is reported for what it demonstrates about the <i>data</i> , not the operator.	42

Acronyms

LLM	Large Language Model
MCP	Model Context Protocol
HPC	High-Performance Computing
SLURM	Simple Linux Utility for Resource Management
SSE	Server-Sent Events
API	Application Programming Interface
SDK	Software Development Kit
GIL	Global Interpreter Lock
NFS	Network File System
RPC	Remote Procedure Call
JSONL	JSON Lines
CSV	Comma-Separated Values
HDF5	Hierarchical Data Format, version 5
p95	95th percentile
RSS	Resident Set Size
APM	Application Performance Monitoring
eBPF	extended Berkeley Packet Filter
DNS	Domain Name System
UI	User Interface

Chapter 1

Introduction

This chapter motivates the thesis: it describes the workload (fleets of LLM agents on HPC clusters) and the observability gap it creates, states the key insight the design builds on, and lists the contributions and the document structure.

1.1 Context: Multi-Agent LLM Systems on Clusters

Applications built on large language models (LLMs) have changed shape quickly over the last few years. The first generation consisted of single request–response calls: a prompt in, a completion out. The second generation wrapped the model in a loop, the *agent*, in which the model reasons about a goal, invokes external tools, observes their results, and iterates until the task is done [31]. The current generation goes one step further and decomposes a task across *multiple* cooperating agents, each with its own role, context, and conversation, which delegate sub-tasks to one another through structured tool-call protocols such as the Model Context Protocol (MCP) [3]. Multi-agent frameworks make this pattern routine [29]: a planner agent farms work out to worker agents, workers consult specialist agents, and the overall computation is a conversation *between* programs as much as within them.

At the same time, the deployment substrate of these systems is changing. When agents were single processes, a laptop or a single cloud instance was enough. Multi-agent workloads (scientific-workflow assistants, large-scale code analysis, data-processing swarms) instead run on *clusters*: tens to hundreds of nodes, allocated through batch schedulers such as SLURM [32], with several agents per node. Recent work places LLM-agent orchestration directly on leadership-class HPC systems [24] and on the workflow infrastructure that spans them [28]. The result is a genuinely distributed system in the classical sense: many processes on many machines, communicating over a network, with no shared clock and no single point of observation.

This combination is operationally demanding in a way neither ancestor was. LLM agents are non-deterministic, expensive (every turn has a token cost), slow (seconds per model call), and failure-prone in unfamiliar ways. They retry, hallucinate tool arguments, hang on remote calls, and silently degrade; empirical studies of multi-agent LLM systems find such failures to be systematic rather than incidental [7]. Running *fleets* of them across a cluster multiplies these problems by the number of nodes and adds the classical failure modes of distributed infrastructure: slow nodes, network partitions, and stragglers that drag down the whole computation [11]. An operator needs to answer questions that span both layers at once: *What is the swarm doing right now? Which agents are talking to which? Why is this scenario slow, and which hop is the bottleneck? Which node is dragging the fleet down? Has this failure happened before?*

This setting is the object of study of this thesis: a fleet of LLM agents distributed across the nodes of a high-performance-computing (HPC) cluster, and the observability infrastructure

required to operate it.

1.2 The Gap: Agent Semantics vs. Cluster Topology

Observability tooling for this setting splits into two mature camps whose intersection is, to our knowledge, unoccupied. The first camp, *LLM and agent observability*, captures rich agent semantics (prompts, tokens, costs, tool calls, rendered as span trees) but treats the physical machines the agents run on as, at best, opaque metadata: host identity is not a dimension the tools can group, compare, or alert over. The second camp, *HPC and fleet monitoring*, is built for that physical layer (per-node metrics at scale, over hundreds of nodes) but carries no application semantics at all: that the process saturating a node is an LLM agent mid-conversation, retrying a failed delegation, is invisible. Section 2.6 surveys both camps in detail, tools and citations included.

The consequence fits in one sentence, which this thesis takes as its motivating problem:

*The tools we survey answer “what did my agent do?” with a span tree of one run on one machine. To our knowledge, none answers “what is my agent **fleet** doing across the cluster, and which node is dragging it down?”*

Answering that question requires coupling agent semantics to physical cluster topology in a single system, the intersection both camps leave open.

1.3 Key Insight and Approach

The technical starting point is an observation about the shape of the data. Everything worth capturing about a multi-agent system is an *event*: an LLM turn (prompt, response, tokens, latency, cost), an agent-to-agent call (caller, callee, correlation, outcome), a context-memory operation (a node added to or evicted from an agent’s working context), a recovery or backtrack. These events carry timestamps and causal relationships: a delegation *happens before* the turns it triggers, which happen before the reply edge. A multi-agent execution, in other words, *is* a causally-ordered event log in the sense of Lamport [17].

If the data is natively a distributed event log, the natural storage substrate is a *distributed shared log* [5, 15]. This thesis builds on ChronoLog [14], a distributed, tiered shared-log store that orders events by physical time and, crucially for this setting, places a log-capture process (a *keeper*) on every cluster node. Agents write locally on the node where they run; the log provides global ordering, durability, and tiered archival; and every observability view is a *reader* over the same substrate. Ordering and durability are properties of the storage layer, not of the instrumentation, and cross-agent causal reconstruction comes with the substrate rather than being bolted on.

On this foundation the thesis designs, implements, and validates ChronoLog Observability: a drop-in observability system for LLM-agent fleets on HPC clusters. Two capture paths feed the log, one for LLM turns and one for inter-agent calls, and a dashboard reconstructs every view (live agent and cluster topology, failure diagnosis, fleet health, cross-host tracing) purely from what was captured. Because a shared log of this class is optimized for ingest (archive reads trail appends by tens of seconds of drain latency, and a live replay can block a reader for minutes), the system adopts a two-path read design: the log is the durable *cold* path, and an in-process *hot* store serves real-time views from the same event stream at ingest time. We validate the system end-to-end with real Claude agents on live multi-node ChronoLog deployments.

1.4 Contributions

This thesis makes four contributions:

1. **A two-path capture architecture over a distributed shared log.** LLM turns flow through a node-local spool drained by a capture worker; inter-agent calls flow through a per-node collector into the node-local keeper. The design provides at-least-once capture and effectively exactly-once ingest via per-session high-water marks, with crash recovery reconstructed from the log itself (Chapter 4).
2. **A hot/cold read design around the log’s ingest-optimized trade-off.** The substrate’s drain latency (tens of seconds) and a replay path that can block the reader for minutes make it unreadable in real time; the system pairs it with an in-process hot store and a server-sent-events (SSE) live bus, so that real-time views and durable full-fidelity views are projections of one event stream (Chapter 4).
3. **Fleet-scale analytics computed on the log.** Three analyses turn the captured events into operator answers: ChronoDoctor, which detects failures, clusters them by signature into ranked incidents, and attaches zero-cost diagnoses with cross-run incident memory; fleet-health rollups that make the cluster overview $O(\text{nodes} \times \text{buckets})$, independent of event volume; and cross-host critical-path tracing that names the bottleneck hop by self-time (Chapter 5).
4. **A measured evaluation on live clusters.** An automated end-to-end harness exercises the full stack with real Claude Agent SDK agents on live 4- and 8-node ChronoLog clusters, and we measure five dimensions: capture overhead, freshness, detection quality against planted ground truth, scalability in event volume and agent count, and a timed cross-layer diagnosis case study (Chapter 6).

1.5 Document Structure

Chapter 2 develops the background (distributed-systems foundations, distributed tracing, the shared-log abstraction, ChronoLog, and LLM agents) and surveys the state of the art to establish the gap. Chapter 3 distills the target scenario into explicit requirements. The system design follows in Chapter 4: the capture paths, the mapping of agent semantics onto the log, delivery semantics, and the hot/cold read split. Chapter 5 builds the fleet-scale analyses on that substrate. Chapter 6 measures overhead, freshness, scalability, and end-to-end correctness, and closes with a fault-localization case study. Chapter 7 concludes.

Chapter 2

Background and State of the Art

This chapter develops the concepts the thesis is built from, as a progression: the distributed-systems fundamentals that define the problem’s vocabulary (§2.1); distributed tracing, the discipline that first made cross-machine causality observable (§2.2); the distributed shared log, the storage abstraction this thesis adopts as its substrate (§2.3); ChronoLog, the concrete log store used (§2.4); LLM agents and multi-agent systems, the workload being observed (§2.5); and finally the state of the art, whose two disjoint camps define the gap this thesis targets (§2.6). Most sections close by noting what the thesis takes from them.

2.1 Distributed Systems Fundamentals

2.1.1 Time, Ordering, and Causality

A distributed system is a set of processes on separate machines that communicate only by exchanging messages. Its defining difficulty is that there is no global “now”: each machine has its own clock, clocks drift, and no observer sees the whole system at once. Any statement about what the system *is doing* must therefore be reconstructed from locally recorded observations.

Lamport’s insight is that such reconstruction turns not on wall-clock simultaneity but on *causality*, captured by the *happened-before* relation [17]: event a happened before event b if they occur in that order in the same process, or if a is the sending of a message that b receives, or transitively through a chain of such steps (Figure 2.1). Happened-before is a partial order (some event pairs are simply concurrent), and it is the order that determines whether one event could have influenced another: in Figure 2.1, a happened before d through the chain $a \rightarrow c \rightarrow e \rightarrow d$, while b and e are concurrent: no causal path connects them, even though b occurs later in wall-clock time.

Logical and vector clocks [17, 10, 22] make this order computable from local information; alternatively, when all events carry timestamps from reasonably synchronized physical clocks and land in one ordered store, causal chains follow from timestamps plus application-level correlation identifiers.

The corollary this thesis leans on is that the *event log*, an append-only sequence of timestamped, attributed events, is the canonical representation of distributed execution. A log captures exactly what each process observed and when; every richer structure (a request trace, a communication graph, a health summary) is a fold over it.

Used in this thesis: multi-agent executions are treated as partially ordered event logs; causal structure is recovered from physical-time ordering plus explicit correlation identifiers carried by the instrumentation (Chapters 4 and 5).

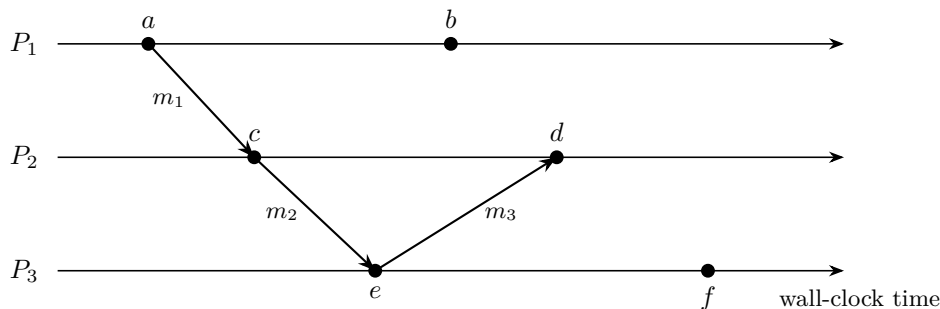


Figure 2.1: A space-time diagram of three processes exchanging messages m_1 – m_3 . Causality flows along process lines and messages: $a \rightarrow c \rightarrow e \rightarrow d$ is a causal chain, so a *happened before* d ; b and e are *concurrent* (neither could have influenced the other), despite b being later on the wall clock.

2.1.2 Delivery Semantics

Any pipeline that moves events from producers to a store must define what happens when a component fails mid-transfer. Three contracts are standard [13]. *At-most-once* delivery never retries, so failures lose events, which is unacceptable for an audit trail. *At-least-once* delivery retries until acknowledged, so failures duplicate events. *Exactly-once* delivery is the ideal, unachievable in general between separately failing components; real systems approximate it as *effectively exactly-once*: at-least-once transport paired with deduplication at the consumer, using idempotent writes or monotonic sequence numbers (“high-water marks”) so that redelivered events are recognized and dropped.

A related design decision is where events wait when a downstream component is slow or dead. Buffering writes in a local, durable *spool* decouples the producer from downstream availability: the producer’s fast path is a local append, and a separate drain process moves data onward at its own pace, the same write-ahead pattern that underlies journaling file systems and message brokers.

Used in this thesis: capture is at-least-once from a durable local spool; ingest into the log is made effectively exactly-once by per-session high-water marks, with consumer state rebuilt from the log itself after a crash (§4.3).

2.2 Distributed Tracing

2.2.1 Lineage: From Dapper to Pixie

Distributed tracing is the discipline of reconstructing, from per-process records, the cross-machine causal story of a single request. Google’s Dapper [27] established the modern vocabulary: a *trace* is a tree of *spans*, each span a timed operation carrying the trace’s identifier, its parent’s identifier, and annotations. Two Dapper design tenets recur throughout this thesis. First, instrumentation must be low-overhead and effectively invisible to application developers, or it will be turned off or bypassed; Dapper achieved this by instrumenting shared infrastructure (the RPC library) rather than applications. Second, complete capture at scale is considered infeasible, so Dapper samples aggressively (uniformly, in its default scheme).

Subsequent systems relaxed different assumptions. The Mystery Machine [8] showed that when explicit propagation is unavailable, the causal model itself can be *inferred*: from large numbers of logged executions, it hypothesizes happened-before relationships and prunes those contradicted by observed orderings. Pivot Tracing [20] made tracing *dynamic*: operators install queries at runtime that propagate context along the execution’s causal path (the “baggage” abstraction), answering questions that were never pre-instrumented. Canopy [12] scaled the

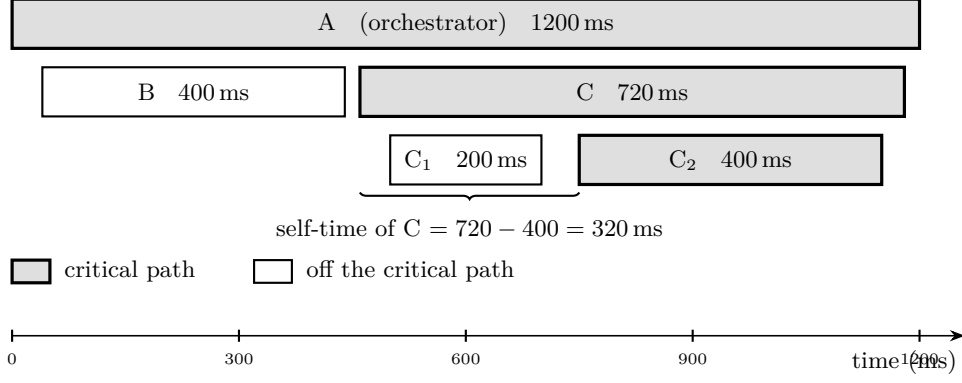


Figure 2.2: A trace rendered as a waterfall. Grey spans form the critical path $A \rightarrow C \rightarrow C_2$, the chain that determines end-to-end latency; white spans (B , C_1) overlap it and cannot shorten the request. Each hop is charged its *self-time*: C ’s 720 ms minus the 400 ms covered by its slowest child leaves 320 ms of C ’s own.

pipeline end-to-end at Facebook, emphasizing that raw traces are not what engineers consume: traces are transformed into feature-indexed datasets that back purpose-built views. More recently, systems such as Pixie [25] pushed capture into the platform itself via eBPF, trading application-level semantics for zero-touch deployment.

2.2.2 Critical-Path Analysis and Self-Time

A trace answers “what happened”; the follow-up question is “why was it slow.” In a tree of overlapping timed operations, total latency is not the sum of span durations: children execute inside their parents, and siblings overlap. The *critical path*, an idea imported into parallel-program performance analysis by Yang and Miller [30], is the chain of operations from root to leaf whose durations determine end-to-end latency: shortening any operation off the critical path cannot speed up the request. Within the critical path, the correct measure of each hop’s contribution is its *self-time* (exclusive time): wall-clock time spent in the operation itself, excluding intervals covered by its children. Attributing latency by inclusive duration double-counts every level of the tree; self-time attribution names the hop actually responsible. Figure 2.2 illustrates both ideas on a small trace: shortening B or C_1 cannot speed up the request, and although C ’s inclusive duration is 720 ms, only 320 ms of it is C ’s own. The Mystery Machine used this analysis to find optimization opportunities at Facebook [8].

Used in this thesis: the tracing view of Chapter 5 reconstructs cross-host call trees from inter-agent events (using explicit parent correlation when present and Mystery-Machine-style inference from session and time nesting when not), computes the critical path, and names the bottleneck hop by self-time.

2.2.3 Sampling and Its Limits at Fleet Scale

Dapper’s sampling assumption does not transfer cleanly to the LLM-agent setting. Head-based sampling (deciding at trace start) keeps coherent traces but misses rare failures by construction. Tail-based sampling (deciding at trace end) must buffer every trace until its fate is known. At fleet scale that reintroduces the data-volume problem sampling was meant to solve, and it requires reassembling spans scattered across many hosts before the keep/drop decision can be made. LLM-agent workloads change the calculus in two ways. First, event rates are modest: an agent turn takes seconds and costs real money, so a fleet of hundreds of agents emits

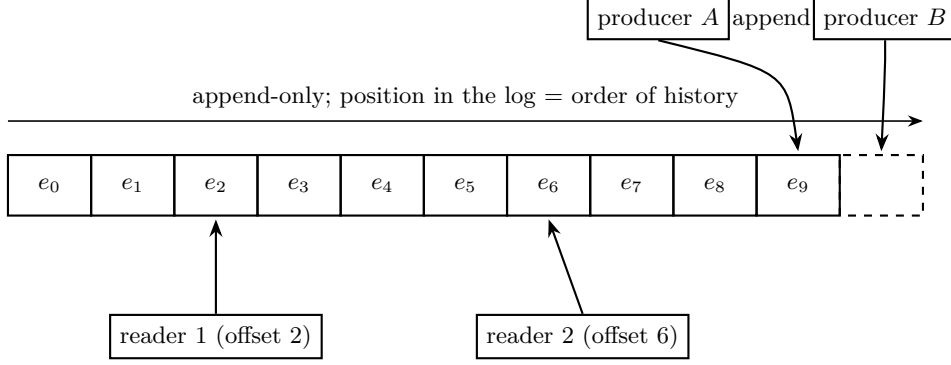


Figure 2.3: The shared-log abstraction. Producers append at the tail; an event’s position in the log is its permanent place in history. Readers consume independently at their own offsets, imposing no back-pressure on producers or on each other.

orders of magnitude fewer events than a microservice fleet emits RPCs¹; complete capture is affordable. Second, the value of completeness is higher: each event is semantically rich (a full LLM interaction), failures are the rare events sampling discards, and post-hoc analyses such as cost accounting require totals, not estimates.

Used in this thesis: the system captures completely and treats storage, not sampling, as the scaling lever, pushing the volume problem down to the log substrate and the read-side rollups (§2.3, Chapter 5).

2.3 The Distributed Shared Log Abstraction

The systems of §2.2 all separate capture from storage: spans are generated in-process, then shipped to a backend whose design is incidental to the tracing model. A complementary line of work treats the *log itself* as the system’s central abstraction. Kreps’s formulation [15] argues that an append-only, totally ordered log is the natural integration point for distributed data systems: producers append, consumers read at their own pace, and the log’s order *is* the system’s notion of time (Figure 2.3). Kafka [16] industrialized this as partitioned commit logs (replication followed in later versions); CORFU [5] showed that a shared log can be a first-class distributed primitive: clients append to a global address space striped over a cluster, and the shared log’s total order becomes the foundation for replicated state machines and transactional systems built above it.

Three properties of this abstraction matter for observability. First, *node-local appends*: producers write to a nearby log segment, keeping the fast path off the network’s critical path. Second, *decoupled readers*: consumers impose no back-pressure on producers; an expensive analysis reads the same log as a cheap dashboard without perturbing capture. Third, *durability with order*: once appended, an event’s position, its place in history, is permanent, the property an audit trail of agent behavior needs.

The abstraction also has a characteristic trade-off: log stores are *write-optimized*. Appends are fast because data lands in ingest-side buffers that are sealed, ordered, and moved to read-optimized storage asynchronously; a reader wanting the most recent events must either wait out this pipeline or read from a separate, freshness-oriented structure. Stream-processing architecture has long formalized the resulting design: the *lambda architecture* pairs a durable, complete, high-latency batch path with a fast, approximate speed layer over the same event

¹The arithmetic: a 100-agent fleet at one turn per agent every few seconds produces tens of events per second; RPC tracing operates against request rates many orders of magnitude higher, which is why Dapper samples at all.

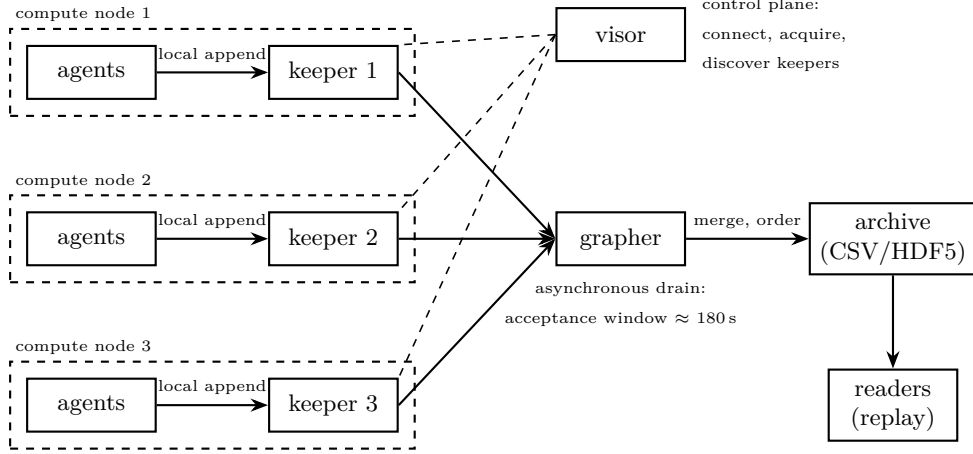


Figure 2.4: ChronoLog’s architecture. A keeper on every compute node takes node-local appends into memory (the ingest tier); the visor coordinates clients; the grapher asynchronously drains, merges, and time-orders sealed chunks into a read-optimized archive. The drain imposes an acceptance window during which fresh events are not yet readable (20–30 s measured end-to-end here, with a 180 s window configured at the grapher). This is the property the read path of Chapter 4 is designed around.

stream [21]. This trade-off, in ChronoLog’s concrete form, drives a central design decision of this thesis (§2.4.3, Chapter 4).

Used in this thesis: the shared log is the single substrate (capture appends to it, every view reads from it), and its write-optimized/read-lagging nature is handled by a lambda-style hot/cold split rather than by abandoning the abstraction.

2.4 ChronoLog

ChronoLog [14] is a distributed, tiered shared-log store developed for HPC environments, and the concrete substrate of this thesis. The paper presents the design; the component names, tier formats, and configuration described below reflect the deployed version used in this work. It differs from the logs of §2.3 in one decision that makes it particularly suited to this thesis’s workload: events are ordered by *physical time* at capture, rather than by a sequencer-assigned logical position, so the log’s global order directly reflects when things happened across the cluster.

2.4.1 Architecture: Visor, Keepers, Grapher–Player

A ChronoLog deployment (Figure 2.4) comprises a coordinating *visor*, a recording group of *keepers*, and an archival pipeline of *grapher* and *player* processes.

The visor is the control plane: clients connect to it to create and acquire logs and to discover the recording group. The keepers are the ingest plane: one keeper runs on every node of the deployment, and clients append events to a node-local keeper, which buffers recent history in memory. Asynchronously, the grapher drains sealed chunks from the keepers, merges them into time order, and writes them to a read-optimized archive (CSV/HDF5 tiers), which the player serves back to readers. The design mirrors the tiered storage hierarchies of HPC I/O systems: a fast, distributed, memory-resident ingest tier in front of a durable, ordered archival tier.

Two consequences of this architecture are load-bearing for this thesis. The keeper-per-node layout means capture is *physically co-located* with the workload: an agent’s events are appended on the machine where the agent runs, and the mapping from events to nodes (the coupling of

semantics to topology that Chapter 1 identified as the gap) is native to the substrate rather than reconstructed. And the drain pipeline means the archive is complete and ordered but *late* (§2.4.3).

2.4.2 Data Model: Chronicles, Stories, Events

ChronoLog organizes data hierarchically. A *chronicle* is a named log namespace; within it, a *story* is an independently written and read event sequence; an *event* is a timestamped payload appended to a story. Clients acquire a story handle to append or replay, and stories provide the unit of isolation: writers to different stories do not contend, and a reader replays exactly one story’s history in time order. The model maps naturally onto observability data: chronicles partition by data kind, stories by session or entity, and events carry the individual observations (§4.2 details this thesis’s schema).

2.4.3 Design Point: Ingest-Optimized, Delayed Reads

ChronoLog’s design point is high-throughput, low-latency *ingest*: appends land in the node-local keeper’s memory and return. The price is paid on the read side, in two forms that this thesis measures and designs around. First, the keeper→grapher drain imposes an *acceptance window* during which freshly appended events are held in ingest buffers and are not yet readable from the archive: in the deployments used here the keeper seals 10s chunks under a 15s acceptance window, for a measured end-to-end availability of 20–30s (§6.3); the grapher and player are configured with a longer, 180s window. Second, a live replay can be not merely stale but *blocking*: a read that the player cannot answer holds the client (and, in a Python client holding the global interpreter lock, the entire calling process) for the full ~180s query timeout. A reader architecture built naively on such a substrate freezes exactly when it is most interesting: while the system is live.

This is not a defect but the standard write-optimized trade-off of §2.3 in concrete form, and it dictates the read-side architecture of Chapter 4: ChronoLog serves as the durable, complete cold path, while real-time views are fed by a hot path populated at ingest. That is the lambda split, with the acceptance window as the boundary between the layers.

2.5 LLM Agents and Multi-Agent Systems

2.5.1 The Agent Loop, Tool Use, and Delegation

An LLM *agent* is a program that couples a language model to an environment through a loop (Figure 2.5a): the model is prompted with a goal and a transcript of prior steps; it emits either a final answer or an *action*, a call to an external tool; the tool’s result is appended to the transcript; and the loop repeats. The pattern was crystallized by ReAct [31], which interleaves free-form reasoning with structured actions. The unit of execution is the *turn*: one model invocation, with its prompt, response, latency, and token consumption.

Tool interfaces are standardizing around protocols such as the Model Context Protocol [3], in which tools are typed, discoverable endpoints. Once tools are first-class, *agents themselves can be tools*: one agent’s toolset includes a call-remote-agent operation that opens a sub-conversation with another agent, possibly on another machine. This is how multi-agent systems form [29]: a scenario is a set of agents, each a loop over its own context, connected by delegation edges. From a distributed-systems standpoint (§2.1.1), a scenario execution is a set of per-agent event sequences joined by inter-agent messages: precisely a partially ordered event log, with delegation calls as the cross-process edges.

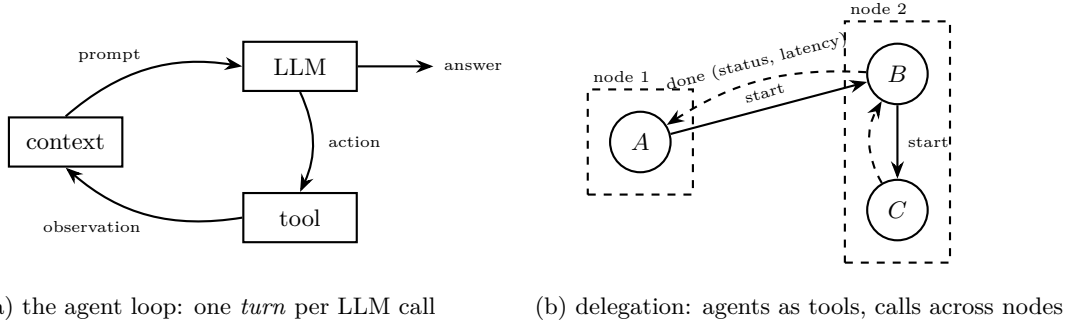


Figure 2.5: (a) The agent loop: each cycle through the LLM is one *turn*, appending the tool’s observation to the agent’s context. (b) Multi-agent delegation: once agents are tools, agent *A* calls agent *B* (possibly on another node), which calls *C* in turn, each call a **start** with a matching **done** (dashed) carrying status and latency. The result is a partially ordered event log with delegations as its cross-process edges.

2.5.2 Observable Signals

The observable surface of an agent fleet has two planes. The *intra-agent* plane is the sequence of LLM turns: prompt and response content, model identity, latency, token counts, and (unusual among workloads) direct monetary cost per event, since providers bill per token. It also includes the agent’s *working memory*: the context assembled for each turn, which grows as results accumulate and shrinks under eviction and summarization when the model’s context window fills. Memory dynamics are an operational signal in their own right: an agent nearing its context limit degrades before it fails. The *inter-agent* plane is the set of delegation calls: which agent called which, when the call started and finished, whether it succeeded, and how calls nest. This is the plane from which scenario structure, communication topology, and cross-host traces are reconstructed.

2.5.3 Failure Modes of LLM Agent Fleets

Agent fleets fail in ways that combine both planes. Empirical taxonomies of multi-agent LLM failures [7] find the dominant categories to be specification gaps, inter-agent misalignment, and missing task verification, all of which surface to an operator as the concrete symptoms below. On the intra-agent plane: provider errors (HTTP 4xx/5xx, rate limits), latency outliers against a model’s typical distribution, and *retry storms*, in which an agent re-issues a failing prompt in a loop, a classic amplification pathology (the retry-driven feedback loops of metastable failure [6]), here with a direct monetary cost attached to every retry. Agents may also *backtrack*, restoring earlier context after an unproductive path; frequent recoveries signal a struggling agent. On the inter-agent plane: failed delegations, and, particularly damaging, *hung* calls (termed *orphan calls* from Chapter 5 on), in which a delegation starts but never completes, silently stalling the delegating agent; in a per-edge view nothing is visibly wrong, since the signature is an absence. At fleet scale these interact with infrastructure pathology: a degraded, fail-slow node [11] drags down every agent scheduled on it, producing a symptom that is scattered across scenarios yet has a single physical cause. Localizing such failures is the cross-layer question neither tooling camp of §2.6 can answer alone.

2.6 State of the Art and the Gap

2.6.1 LLM and Agent Observability

The first camp instruments the application layer. LangSmith [18] (from the LangChain ecosystem), Langfuse [19], and Arize Phoenix [4] capture LLM calls and tool invocations as hierarchical runs or OpenTelemetry-style spans, adding LLM-specific value: token and cost accounting, prompt management, dataset curation, and LLM-as-judge evaluation. AgentOps [2] targets agent frameworks specifically, tracking sessions, tool usage, and multi-agent hand-offs. General application-performance-monitoring (APM) vendors have added LLM-observability products [9], and the OpenTelemetry project is standardizing generative-AI semantic conventions [23], signalling consolidation of this camp around the span model.

The limitation is structural, not incidental. The span model these tools inherit was designed for request-scoped microservice tracing: “distributed” means span correlation across services in one application’s request path; “multi-agent” means multiple agent abstractions within one process or one orchestrator. Host identity, where it appears at all, is opaque metadata: a tag an operator can attach and read back, not a first-class dimension the tools can group by, compare across, or alert over. Population-level questions (compare nodes, find the sick one, correlate agent failures with machine placement) therefore cannot be phrased. The nearest exceptions prove the shape of the gap rather than filling it: an APM platform such as Datadog can correlate an LLM trace with the host metrics of *that request’s* infrastructure, but offers no fleet-of-agents semantics over an HPC allocation: no batch-scheduler awareness, no population view keyed by agent behavior. The OTel generative-AI conventions standardize *turn* semantics, while the host resource attributes an SDK may attach come with no defined fleet-level analysis. In both cases the data model cannot express the join the operator questions of §1.1 require: from a fleet-wide symptom to a specific agent’s conversation on a named machine.

2.6.2 HPC and Fleet Monitoring

The second camp instruments the physical layer, and does so at scale. Prometheus [26], typically paired with Grafana dashboards, is a de facto standard for metrics-based monitoring, routinely deployed over SLURM clusters via node exporters and job accounting; HPC-native systems such as LDMS [1] are engineered for continuous, low-overhead collection of node metrics across very large machines. These systems answer per-node and per-job questions (utilization, saturation, hardware health, stragglers) well, and their visual idioms (heatmaps, per-node grids) are built for hundreds of nodes.

Their blindness is the mirror image of the first camp’s. The data model is metric time series named by host and counter; there is no application semantics at all. That a CPU-saturated node is running an LLM agent whose retry storm is the actual pathology (and that the same storm is visible as cost and latency in some scenario’s transcript) is invisible: metrics cannot say *which conversation* is responsible, and offer no path from a red node to the agent behavior causing it.

2.6.3 The Empty Intersection

Figure 2.6 summarizes the landscape along its two axes: richness of agent semantics, and awareness of physical fleet topology. Camp 1 occupies the semantics-rich, topology-blind quadrant; Camp 2 the topology-aware, semantics-blind one. The quadrant combining both (agent semantics coupled to cluster topology, at fleet scale) is, among the tools surveyed here, unoccupied. It is where the operator questions of §1.1 live: they require pivoting *between* layers, from a fleet-level symptom to a single agent’s conversation and back.

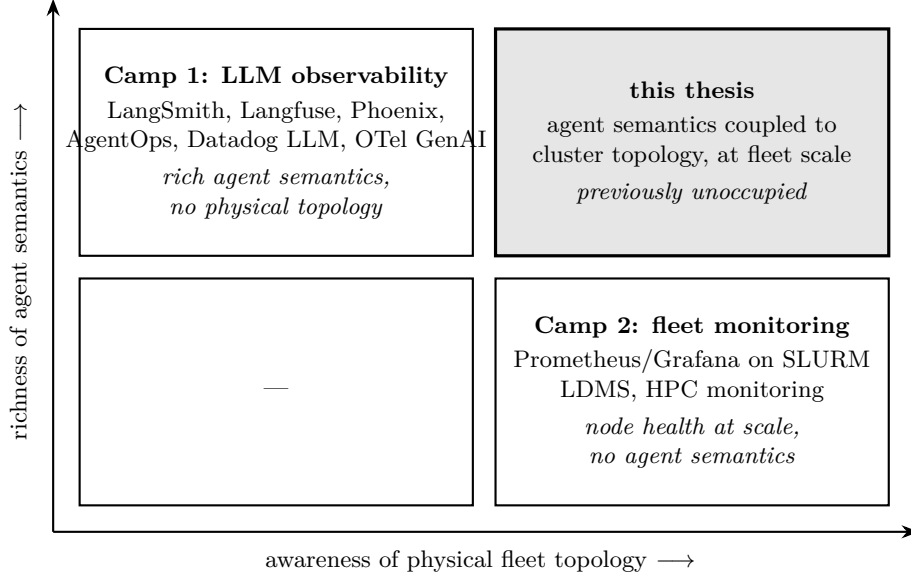


Figure 2.6: The observability landscape for multi-agent LLM systems on clusters, along its two capability axes. Camp 1 and Camp 2 each occupy one off-diagonal quadrant; their intersection (agent semantics coupled to physical fleet topology) is unoccupied among the surveyed tools, and is the quadrant this thesis targets.

Filling that quadrant is not a matter of running both camps side by side: two disjoint systems with disjoint data models provide no join between an agent event and a node, which is precisely the pivot the questions require. The evaluation returns this structural argument as a measurement (§6.7): faced with an injected cross-layer fault on a live cluster, an operator given the raw captured files (all the signals both camps would collect, join included) could not produce a single localization answer. Through the joined views, that operator completed the full symptom-to-conversation pivot in 72 seconds. The same experiment also separates the gap’s two halves: sufficiency is a property of the *data model* (an expert reader extracted every answer from the log files alone, because host attribution and causal correlation are in the events themselves; §6.7 gives the details), while accessibility is a property of the *interface* built on that model. The camps fail at the first half, which no interface can repair: a view over data that lacks the join cannot invent it. The gap therefore calls for a single substrate in which agent-semantic events are natively attributed to physical nodes: what the shared-log approach of §2.3, instantiated on ChronoLog’s keeper-per-node architecture (§2.4), provides by construction. Designing the capture, read, and analysis layers that turn that substrate into a working observability system is the subject of the remainder of this thesis.

Chapter 3

Problem Statement and Requirements

This chapter turns the gap established in Chapter 2 into a concrete engineering problem: a target scenario (§3.1), a set of requirements that any solution must satisfy (§3.2), and the boundaries of what this thesis does not attempt (§3.3). The requirements are numbered R1–R7 and are used as the evaluation rubric in Chapters 6 and 7.

3.1 Target Scenario

The setting is a multi-agent LLM workload on an HPC cluster. A user holds a SLURM allocation of M compute nodes (in this thesis’s experiments, $M \in \{4, 8\}$ on the ARES cluster, with analyses exercised up to several hundred simulated nodes). On each node run K LLM agents: independent processes, each an agent loop over its own conversation context, each identified by a scenario-scoped session identifier of the form `role@scenario`. Agents cooperate: an agent may delegate a sub-task to a peer on another node by invoking a remote-agent MCP tool, producing nested chains of cross-host calls. A ChronoLog deployment shares the allocation: a keeper on every node, a visor coordinating, and a grapher draining sealed chunks to an archive on shared storage.

The human in the loop is the *operator*, the researcher or engineer running the fleet, who needs answers both while the system runs and after it finishes:

- Q1. *What is the swarm doing right now?* Which agents exist, which are talking to which, and on which physical nodes?
- Q2. *Is anything failing?* Not as a scroll of per-event errors, but as a ranked, deduplicated set of problems with causes.
- Q3. *Why is this scenario slow?* Which hop in a cross-host delegation chain is the bottleneck?
- Q4. *Which node is dragging the fleet down?* A population-level comparison across all M nodes, including silent ones.
- Q5. *Has this happened before?* Whether a failure is novel or a recurring pattern across runs.

Q1–Q3 are agent-semantic questions; Q4 is a fleet-topology question; Q5 is a longitudinal question. Q2–Q4 in combination require pivoting between layers (from a fleet symptom to a single agent’s conversation), the pivot neither tooling camp of §2.6 supports.

3.2 Requirements

R1: Dual-plane causal capture. The system must capture both observable planes of §2.5.2: LLM turns (prompt, response, model, tokens, cost, latency, working-memory deltas) and inter-agent calls (caller, callee, hosts, timing, outcome). The two planes must also be *joinable*: an inter-agent edge must resolve to the conversations of its endpoint agents, and every event must be attributed to the physical node that produced it. Without the join, Q1 and the Q2–Q4 pivot are unanswerable.

R2: Real-time visibility. Live views (Q1, and the live half of Q2/Q4) must reflect events within seconds of ingest. In particular, the system must not be limited by the substrate’s drain latency (§2.4.3), and no read may ever block the serving process on un-drained log state.

R3: Durability and replayability. Every captured event must survive process and node restarts and remain replayable after the run ends: post-hoc analysis, cost accounting, and the longitudinal question Q5 all read history. Transient failures of downstream components must not lose events (at-least-once capture), and retries must not duplicate them in the durable store (effectively exactly-once ingest, §2.1.2).

R4: Fleet scale. Population-level views (Q4) must remain responsive at 100+ nodes. Concretely, the cost of rendering the fleet overview must not grow with the number of captured events, only with the number of nodes and the length of the queried window. Silent nodes must also appear, since silence is itself a signal.

R5: Minimal, non-blocking overhead. Instrumentation must not perturb the workload it observes (the Dapper tenet, §2.2.1). The agent-side fast path must be a local, constant-time operation (an append to node-local state), with all network and log I/O off the agent’s critical path. The capture machinery must degrade gracefully (buffer, fall back) and never back-pressure an agent.

R6: Zero-marginal-cost diagnosis. Failure detection and diagnosis (Q2, Q5) must not itself consume LLM tokens by default: a diagnosis pipeline that costs money per scan will be disabled in practice. LLM-assisted diagnosis may exist, but only as an opt-in on top of a deterministic baseline.

R7: Drop-in deployment. The system must attach to an existing ChronoLog deployment without modifying ChronoLog, the scheduler, or the agent framework: instrumentation via a thin client library or local HTTP endpoint, per-node components launchable on a plain SLURM allocation, and a read-only dashboard that can point at any visor.

3.3 Non-Goals

Four adjacent problems are explicitly out of scope. The system is not an *agent framework*: it observes agents built with existing SDKs and protocols, and takes no part in orchestrating them. It is not a *scheduler or resource manager*: placement, allocation, and job control remain SLURM’s, and the system only queries it. And it is not a general *metrics time-series database*: it captures the agent-semantic event planes of R1, not arbitrary node telemetry. Where classical infrastructure metrics are needed, the systems of §2.6.2 remain the right tool, and the two can run side by side. Finally, the thesis targets observation and diagnosis, not automated

remediation: acting on a diagnosis (restarting agents, rolling back context) is surfaced as a recommendation to the operator, not executed autonomously.

With the questions, requirements, and non-goals fixed, the problem is fully specified. Chapter 4 presents a design that meets R1–R7 on the ChronoLog substrate; Chapter 6 then measures each requirement against it.

Chapter 4

System Design

This chapter presents the design of ChronoLog Observability. Section 4.1 gives the architecture: two capture paths converging on one shared-log substrate. Section 4.2 defines how agent semantics map onto the log’s data model. Section 4.3 specifies the write path and its delivery guarantees. Section 4.4 addresses the substrate’s read latency with a hot/cold split. Section 4.5 covers process and failure isolation, and §4.6 the extensibility seam.

4.1 Overview: Two Paths, One Substrate

Every datum enters the system through one of two ingest paths (Figure 4.1), both terminating in ChronoLog via the keeper on the producing node.

Path-A carries the intra-agent plane: LLM interactions (one record per turn), context-graph deltas (one record per working-memory add or eviction), and recovery events. Producers call a small capture library that appends the record to a node-local, append-only *spool*; a separate *capture worker* process periodically drains the spool into ChronoLog stories.

Path-B carries the inter-agent plane: agent-to-agent MCP calls. Each logical call is emitted as a *pair* of events sharing a correlation identifier: a **start** when the delegation is issued and a **done** when it completes, the latter carrying status and latency. Agents POST these events to a *collector* daemon on their own node (`localhost:5650`); the collector writes them to the node-local ChronoLog keeper and simultaneously forwards a thin copy to the dashboard, which feeds the live views (§4.4).

The two paths differ deliberately. Path-A events are large (full prompts and responses), produced at low rate (one per turn, i.e. per LLM call), and consumed mostly after the fact, so a durable local buffer plus a batch drain fits. Path-B events are small, structurally fixed, and latency-relevant (the live scenario graph is drawn from them), and they define the cross-agent causal structure; a daemon that stamps, persists, and immediately fans them out is the better match. What the paths share is the destination and the discipline: node-local append on the producing machine (R5), durable persistence in the log (R3), and host attribution stamped at source (R1).

The dashboard itself is a *pure reader*: a Flask application serving a React single-page app that reconstructs every view from ChronoLog and the hot store, and captures nothing on its own (§4.5).

4.2 Mapping Agent Semantics onto the Log

ChronoLog organizes data as chronicle $\rightarrow$ story $\rightarrow$ event (§2.4.2). The mapping adopted here (Table 4.1) uses chronicles to partition by *data kind* and stories by *entity*, so that every read pattern of Chapter 5 replays exactly one story:

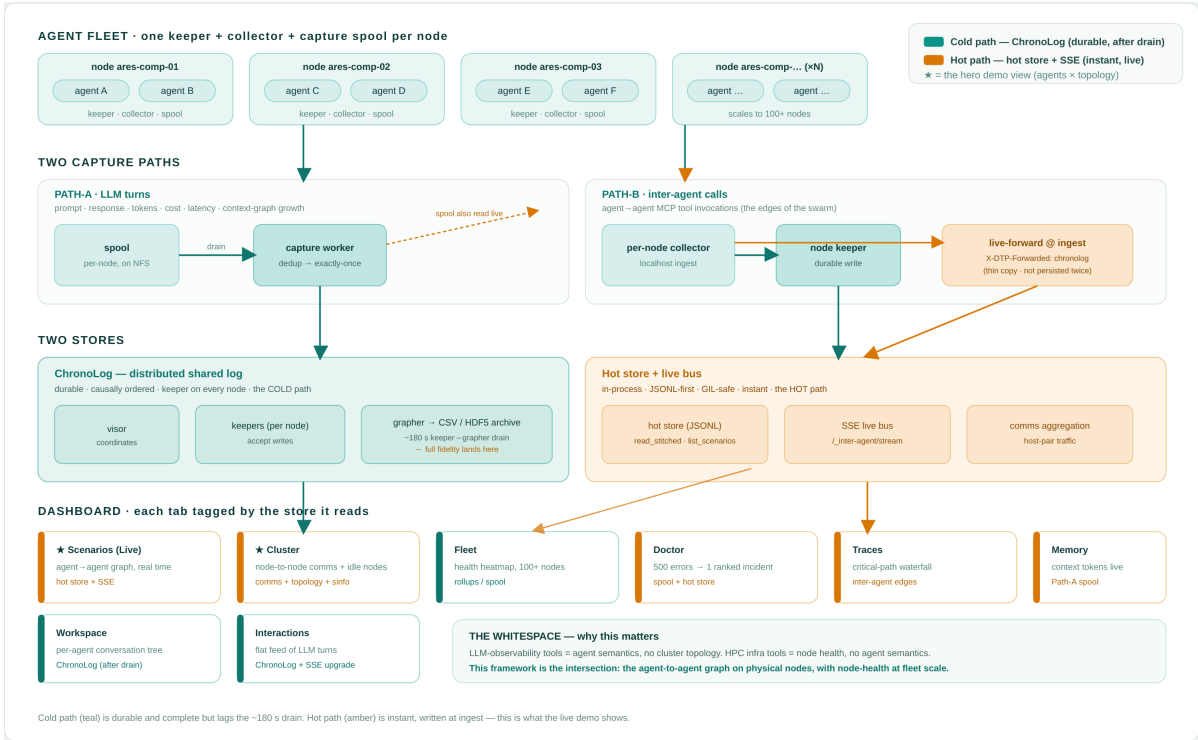


Figure 4.1: The two-path architecture. Path-A carries LLM turns through a node-local spool drained by a capture worker; Path-B carries inter-agent calls through a per-node collector. Both persist to ChronoLog through the node-local keeper; the dashboard reads ChronoLog (cold, full fidelity) and a local hot store (live).

Session identity. The load-bearing convention is the session identifier `role@scenario` (e.g. `writer@scenario-x`). It is simultaneously the ChronoLog story name for all Path-A data of that agent, the `from_session/to_session` endpoint on Path-B edges, and the join key the dashboard uses when a clicked node in the scenario graph opens that agent’s conversation. Scoping the role by scenario keeps the identifier globally unique across concurrent scenarios while leaving it human-readable. Because the same string names the spool file, the ChronoLog story, and the graph node, the identity round-trips through every tier with no lookup table. This is R1’s “joinable planes” made concrete: the join is by construction, not by correlation heuristics.

Causal structure. Cross-agent causality is carried by two explicit fields on Path-B events. The `correlation_id` stitches the `start/done` pair of one call into a single edge; the optional `parent_correlation_id` records the call being *served* when a new call was issued, giving the tracer exact call trees. When instrumentation does not propagate the parent, the tracer falls back to inference from session identity and time nesting (§5.4), the Mystery-Machine position on the spectrum of §2.2.1, with explicit propagation as the Dapper position when available. Within a session, Path-A records carry a monotonic per-stream `sequence_id`, which orders turns without reference to wall clocks.

Listing without enumeration. One practical gap in the substrate is the absence of a chronicle-enumeration operation in the client binding: there is no way to ask “which scenarios exist?” without knowing their names. The design routes around it with an `__index__` chronicle: every component that first sees a new scenario or session appends a one-line event (`{“v”: <name>}`) to a well-known index story, and listing becomes a replay of that story, deduplicated in

Chronicle	Story	Holds	Path
llm_interactions	<i>session id</i>	LLM turns	A
context_graphs	<i>session id</i>	working-memory deltas	A
recovery_events	<i>session id</i>	recovery/backtrack events	A
ia_<scenario>	edges	inter-agent start/done events	B
metrics_rollup	minute	per-(host, minute) fleet buckets	rollup
__index__	scenarios / sessions	listing index	both

Table 4.1: The chronicle/story schema: chronicles partition by data kind, stories by entity. Session-scoped stories serve the conversation and memory reads; the per-scenario edge story serves graph and trace reads; the rollup and index stories are written by the capture side so that readers never scan.

first-seen order. The index is append-only and idempotent to re-add, so it needs no coordination among the writers. The pattern is a small instance of the log-as-substrate principle: when a query is missing, materialize it as another log.

4.3 Write Path and Delivery Semantics

The write path is engineered backwards from R5 (never perturb the agent) and R3 (never lose, never duplicate).

The spool as write-ahead buffer. A Path-A producer’s fast path is an append of one JSON line to a per-(session, kind) spool file, opened with `O_APPEND` so that concurrent writers interleave atomically at line granularity. No network, no lock, no dependency on any downstream component being alive: if ChronoLog, the capture worker, or the dashboard is down, agents keep running and the spool absorbs the backlog. The capture library injects and guarantees exactly three fields: `session_id`, a monotonic `sequence_id` (seeded from the maximum found on disk, so monotonicity survives producer restarts), and a `timestamp`. It treats the rest of the record as an opaque producer contract.

Effectively exactly-once ingest. The capture worker drains the spool into ChronoLog on a polling interval. Transport is at-least-once by construction (after a crash the worker re-reads spool files it may have partially drained), so ingest must deduplicate. The worker keeps a *high-water mark* per (session, kind) stream: the largest `sequence_id` already written to ChronoLog. Only records above the mark are appended. The critical property is where that state lives: on startup the worker *rebuilds the marks from ChronoLog itself*, replaying each session’s stories to find the highest sequence present. The log is thus the single source of truth for what has been ingested, and the worker carries no durable state of its own. No worker crash, restart, or migration to another node can cause loss (the spool retains everything) or duplication (the rebuilt marks skip everything already present). Records without a natural sequence (recovery events) carry a unique blob identifier and are deduplicated by set membership, seeded the same way. The arrangement is the textbook recipe of §2.1.2 (at-least-once transport plus idempotent, monotonic ingest), instantiated with the log as its own ledger.

Path-B: persist once per backend. The collector gives Path-B events the same guarantees with one extra subtlety. Every event travels to *two* places: the node-local keeper (durable) and the dashboard (for the live bus). Naively this double-writes. The design makes the forward explicit: when, and only when, the collector has successfully written an event to ChronoLog, it sets a header (`X-DTP-Forwarded: chronolog`) on the forwarded copy. The dashboard

honors the header by preserving the collector-stamped identifiers verbatim and skipping its own ChronoLog write, storing only the hot copy. If the keeper write failed, the collector forwards *without* the header and the dashboard persists the event through its own backend; the placement is degraded, but the event is not lost. The net contract: each event is persisted exactly once per backend, and the identifiers (`event_id`, timestamps) are identical in both copies, so the live view and the durable view never disagree about identity.

Graceful degradation. One more fallback closes the path: an agent’s `emit()` tries the node-local collector first and falls back to POSTing directly to the dashboard’s ingest endpoint if the collector is down. The fallback sends no forwarded header, so the dashboard persists normally. An agent therefore observes at most a failed local HTTP call before falling back: never a lost edge, and never back-pressure from the log (R5).

4.4 The Read Problem: Hot and Cold Paths

Writing to an ingest-optimized log is easy; reading it live is the hard part. As established in §2.4.3, freshly appended events sit inside the keeper→grapher acceptance window (20–30 s measured end-to-end in the deployments used here) before they are readable from the archive. Worse, a replay the player cannot answer does not return stale data but *blocks* for the client’s ~180 s query timeout, in a Python client that holds the GIL throughout. A dashboard that naively replayed stories on request would freeze for minutes precisely while the fleet is live, a direct violation of R2.

The design response is a lambda-style split (§2.3) with the acceptance window as the boundary:

The cold path is ChronoLog itself: complete, durable, ordered, and authoritative, but late. Full-fidelity views (a session’s complete conversation, post-hoc analysis) read it, either by live replay once a story has drained or from the grapher’s archive files. Reads that risk touching un-drained state are avoided structurally: the listing index of §4.2 is read *archive-first*, because a live replay against a never-written index story blocks indefinitely (§4.5). The evaluation later hardened this rule into a deployment mode: after live replay proved unreliable end-to-end on the cluster, archive-first reading was generalized from the listing index to *every* replay the serving processes issue, bounding staleness at the drain window (§6.3, §6.8).

The hot path serves R2 and is populated *at the moment of ingest*, before ChronoLog acceptance. It has two elements. First, a *hot store*: the ingest endpoint appends every Path-B event to a local JSONL file per scenario, and all live read APIs (the scenario graph, the node-communication view, fleet aggregation, the tracer) read this store, an instant, append-only local read that can never block on the log. Second, a *live bus*: an in-process publish/subscribe fan-out that pushes each newly ingested event to server-sent-event subscribers, so the browser renders edges as they happen rather than polling. Path-A has the analogous arrangement: the Health view and the per-agent memory panels read the capture spool directly (the spool doubling as Path-A’s hot store) while ChronoLog remains the durable copy.

The late-join protocol. A client that opens a live stream mid-run needs history plus continuity without gaps or duplicates. The bus protocol delivers the stored backlog first, then live frames, and the subscription is registered *before* the backlog is read, so an event ingested during the handoff appears in both, and the client deduplicates by event identifier. Delivery to subscribers is intentionally weaker than delivery to storage: per-subscriber queues are bounded and drop oldest under pressure, and a publish never blocks or raises into the ingest path. By

the time an event reaches the bus it is already durable, so the live stream is a best-effort projection of a reliable log, not a second reliability domain (R5).

Consistency model. Each view advertises, in effect, one of two freshness classes: hot views are eventually consistent with the log but fresh to within the ingest latency (measured in Chapter 6); cold views are complete and ordered but stale up to the acceptance window. Because both are projections of the same event stream with identical identifiers (§4.3), the classes converge: after the drain, a hot view’s content is a prefix-complete subset of the cold view’s, and the operator can always escalate from “live picture now” to “authoritative record later” without changing tools.

4.5 Process and Failure Isolation

The deployment decomposes into single-responsibility processes per node (agents, collector, capture worker), plus one dashboard for the allocation. Three isolation rules, each learned from a concrete failure on live clusters, govern the decomposition.

The dashboard is a pure reader. An early configuration ran the capture worker as a thread inside the dashboard process. On startup the worker warms its high-water marks by replaying sessions (§4.3); on a cluster where a story sat inside the acceptance window, that replay blocked while holding the GIL, and the Flask server never finished binding its port. The lesson generalizes: a process that must always answer (the dashboard) may never share an interpreter with a component that may legitimately block on remote state (the capture worker). They now run as separate processes, and the rule extends to reads: no request handler ever issues a replay that could touch un-drained state.

Never block on state that may not exist. A fresh cluster exhibits a boundary case. Replaying an index story that has *never been written* blocks indefinitely: there is no data to drain, and none will ever arrive. Listing reads are therefore archive-first (the archive’s absence is detectable instantly, while a live replay’s futility is not), with live replay behind an explicit opt-in; and a brand-new deployment seeds the index with a one-shot append-only sync pass, sidestepping the blocking warm-up entirely.

Normalize identity at the edge. Host names arrive from multiple authorities (reverse DNS reports `ares-comp-3`, SLURM reports `ares-comp-03`), and any view that joins on host names (fleet health, the cluster graph, allocation membership) silently drops nodes if the two spellings coexist. All host names are therefore canonicalized (digit padding stripped) at every boundary where they enter a data structure. Identity normalization, like time, belongs at the edge of the system, not in each consumer.

Failure containment follows from the decomposition: an agent crash loses nothing already spooled or emitted; a collector crash falls agents back to direct dashboard ingest; a capture-worker crash pauses draining while the spool absorbs; a dashboard crash affects only observation (capture continues); and a ChronoLog outage degrades the system to hot-store-only operation, with the spool and collector queues replaying into the log when it returns.

4.6 Extensibility: Adapters and Capabilities

Every dashboard view is declared as a *capability* (conversations, scenarios, diagnostics, fleet, tracing, ...) served by a *source adapter*, an interface discovered at runtime through a Python entry-point group. The shipped ChronoLog adapter serves all capabilities; at startup the

application discovers active adapters, mounts only the API blueprints some adapter can serve, and advertises the active set to the frontend, which renders only those tabs. The seam exists for third parties: a deployment with a different substrate implements the adapter interface and gains the dashboard unchanged. The same seam is what makes the offline demo mode (local stores, no cluster) a configuration rather than a fork. This chapter’s design is thereby separable: the capture paths, delivery semantics, and hot/cold split of §§4.3–4.4 are ChronoLog-specific engineering; the analytics of Chapter 5 depend only on the capability interface.

Chapter 5

Fleet-Scale Analytics on the Log

Chapter 4 produced a substrate: every observable event of the fleet, durably ordered, attributed to node and agent, readable hot or cold. This chapter presents the analyses that turn that substrate into answers to the operator questions of §3.1. Each section follows the same shape: the question, the algorithm, and the argument for why it stays cheap at fleet scale.

5.1 Live Topology Coupling: Scenario Graph and Cluster Graph

The first question (Q1) is situational awareness: what is the swarm doing, right now, and where. The system answers it with two live graphs, built deliberately as *two projections of the same event stream*.

The *scenario graph* projects Path-B edges onto agent identity: nodes are sessions (`role@scenario`), edges are stitched calls labeled with tool, status, and latency, built incrementally in the browser from the live bus: backlog first, then per-event frames (§4.4). LLM-observability tools already offer this view (§2.6.1); here it is rendered live. The *cluster graph* projects the very same edges onto physical topology: hosts as nodes, aggregated host-pair traffic as edges (counts, error counts, active agents per host), computed by folding the hot store. Because every edge carries `from_host` and `to_host` stamped at capture (R1), the projection is a group-by, not an inference.

The coupling is completed by the scheduler’s view of the machine: the dashboard queries SLURM (`sinfo`) for the allocation and the cluster’s total/idle/busy/down states, draws idle-but-available nodes alongside the active deployment, and uses only real node names throughout. A node with no events is drawn grey, not omitted; at fleet scale, silence is a signal (R4). Clicking through connects the layers: an agent node opens that agent’s conversation (the `role@scenario` join of §4.2); a host cell pivots to the per-node view. This two-projection design (same events, agent layer and machine layer) is the direct answer to the gap of §2.6.3.

5.2 ChronoDoctor: Detection, Clustering, and Diagnosis

The second question (Q2) is not “show me the errors” but “tell me what is wrong”: at fleet scale, a per-event error feed is least readable when it matters most. ChronoDoctor is a five-stage pipeline (*gather* → *detect* → *cluster* → *diagnose* → *remember*) that turns raw events into a ranked list of explained incidents.

Gather and detect. Gathering is the only I/O: interactions and recoveries from the Path-A spool (hot, GIL-safe), stitched edges from the Path-B hot store, with explicit bounds on sessions and scenarios scanned. Detection is a set of pure, unit-tested functions over the

gathered lists, each emitting typed *signals*: HTTP 4xx/5xx and explicit errors on turns (429 classified separately as rate-limiting); latency outliers, flagged relative to the *per-model* median (an absolute threshold would misclassify a slow model as an incident); retry storms (the same prompt re-issued repeatedly within a session); failed inter-agent calls; *orphan calls* (a **start** with no **done**, the signature of a hung or dropped delegation); slow edges relative to the per-tool median; and recovery events. Orphan calls are always ranked critical: the delegating agent is silently stalled, and no per-edge view shows anything wrong. Detection is a single pass: $O(n)$ in gathered events, with per-group medians over small per-model and per-tool groups.

Clustering. The insight that makes the view scale is that fleet-wide failures are *textually self-similar*. Each signal’s message is normalized, with identifiers, hex runs, and digit strings replaced by placeholders (`peer ares-comp-12 timed out after 30001ms` → `peer ares-comp-<n> timed out after <n>ms`), and the pair (kind, normalized message) is hashed into a *signature*, the clustering key. Five hundred identical timeouts across the fleet collapse into one incident with a count of 500; hosts, sessions, tools, models, and scenarios are aggregated *inside* the incident as its blast radius, deliberately excluded from the key so that a fleet-wide failure stays one card while its distribution stays visible. Severity escalates with count, and incidents rank by (severity, count, hosts affected), worst first.

Diagnosis. Each incident gets a diagnosis (root cause, remediation, blast radius, confidence) from a deterministic, first-match-wins rule engine keyed on signal kind and sample text: orphan calls map to hung-peer causes, 429s to rate limiting, transport-layer markers to network-fabric issues, token-overflow patterns to context exhaustion, and so on, each with a calibrated confidence; a conservative fallback catches the rest. The engine costs nothing per scan (R6). An LLM-backed diagnoser can be enabled explicitly, and any failure of it falls back to the heuristic: the deterministic path is the contract, the LLM an enhancement.

Memory. The final stage answers Q5. Every scan appends one line per incident to a Markdown incident memory; when the recent section exceeds a threshold, the oldest lines fold into a digest with running counts; the file is *self-compacting*, so it stays small across arbitrarily many runs while retaining per-signature totals. On each scan, current incidents are matched against remembered signatures, and a repeat offender is badged **recurring** $\times N$. The memory is a plain human-readable file: the operator can read, annotate, or delete it, and it survives restarts by construction.

5.3 Fleet Health at Scale

The fourth question (Q4) is population-level: which node is unlike the others. The node-link rendering of §5.1 answers it only up to a dozen nodes; past that, it is spaghetti. The fleet view therefore changes representation (a heatmap grid, one cell per node, colored by health, worst first) and, more fundamentally, changes the *read complexity*.

Computing node health from raw events is $O(\text{events})$ per refresh, untenable as history grows. The system instead maintains *rollups*: for each (host, minute) pair, a bucket aggregating that minute’s activity (event and error counts, latency sum/count/p95/max, token and cost totals, distinct agents). An emitter recomputes and appends full bucket snapshots to a dedicated rollup story; because each append is a complete snapshot of its cell, the reader keeps the last value seen per key and re-emission never double-counts: idempotence by last-writer-wins, the same discipline as §4.3. A health query then folds only the buckets in the requested window: $O(\text{nodes} \times \text{buckets})$, independent of event volume (R4). Per-node health folds by explicit rules: sums for counts, a ratio for error rate, and the *maximum* of per-minute p95s as the window’s

p95 estimate (an upper-biased estimate that avoids retaining raw samples). Thresholds then classify each node ok/warn/critical. Nodes in the allocation that produced nothing are injected as empty rows, and every row is tagged with its live SLURM state, so the grid always shows the whole population. The emitter is optional. Absent rollups, the API falls back to folding raw events on demand, which is correct at small scale and keeps the tab from ever being empty.

5.4 Cross-Host Critical-Path Tracing

The third question (Q3), why this scenario is slow, is the classical tracing question of §2.2, posed across hosts. The tracer reconstructs call trees from stitched Path-B edges in two steps.

Tree reconstruction. Each stitched edge is a span: an interval on the timeline from **start** to **done**, with endpoints, tool, status, and latency. Parentage is resolved by a two-tier rule mirroring §4.2: if a span carries an explicit `parent_correlation_id`, that is its parent, the Dapper-style ground truth. Otherwise the parent is *inferred*: the span whose callee session matches this span’s caller session and whose interval encloses this span’s (the tightest such enclosure winning), on the Mystery-Machine premise that a delegation issued by agent *B* while *B* was servicing a call must be causally nested within it. Spans with no parent are roots; each root yields one trace, with depth assigned by tree walk.

Critical path and bottleneck. Within a trace, end-to-end latency is attributed by the analysis of §2.2.2: the critical path is walked from the root by descending into the slowest child at each level, and each hop on the path is charged its *self-time* (its latency minus its slowest child’s), so that time spent waiting on a child is charged to the child, not double-counted in the parent. The hop with maximal self-time is named the *bottleneck*, with its hosts and tool: not “the scenario is slow” but “the **writer**→**reviewer** hop on **ares-comp-07**, 4.2s of self-time.” Because hosts ride on every span, the pivot to Q4 is immediate: if the bottleneck’s node also glows in the fleet heatmap, the cross-layer diagnosis (fleet symptom, physical cause) completes in two clicks. Chapter 6 follows exactly this pivot as a case study. The waterfall rendering shows the trace with depth indentation and the critical path highlighted; reconstruction is $O(s^2)$ in the spans of a scenario in the inference case (s small, tens of calls) and $O(s)$ with explicit parents.

5.5 The Operator Surface: Five Views over One Log

Everything in this chapter and the last is, from the operator’s chair, five tabs in one dashboard. Each view is a projection of the same captured event stream (none holds private state that is not reconstructible from the log), and each exists to answer one of the operator questions of §3.1. This section walks the surface as an operator sees it; Figures 5.1 through 5.5 are captures of the deployed system.

Scenarios: who is talking to whom (Fig. 5.1). The dashboard opens here. One scenario is selected from the left rail, its agents drawn as peers grouped into host containers, every inter-agent call an animated edge (color-coded by status, errors in red). The header carries the scenario’s live totals (hosts, agents, messages, errors) and a *Live/Full* toggle: *Live* renders from the hot store and SSE bus as events arrive; *Full* re-reads the drained archive for complete fidelity, including a replay scrubber that re-animates history at chosen speed. Clicking any agent, host box, or edge opens its detail panel; from an agent the operator can pivot into its full conversation (the turns, token counts, and costs behind the node), which is the Q1 join of capture to conversation, exercised end-to-end by the functional harness of §6.6.

Health: what is wrong, and why (Fig. 5.2). The ChronoDoctor surface. The center column is the live LLM-call feed (every turn as it happens, filterable to errors or a provider); the left rail is the ranked incident list a scan produces: one card per clustered incident, ordered by severity, with occurrence counts and *recurring* badges read back from the incident memory. Selecting an incident opens its diagnosis card: *why this is happening* and *how to fix it* in prose, the blast radius (occurrences, hosts, agents, scenarios), the heuristic’s stated confidence, and the raw evidence samples that justify it. The compression argument of §5.2 is legible in the ratio: thousands of raw signals arrive as a dozen ranked cards, each carrying its own justification.

Cluster: agent semantics on physical topology (Fig. 5.3). This view puts the substrate itself on screen: the ChronoLog deployment drawn on the allocation’s nodes (visor, one keeper card per node with the chronicles and stories it holds, grapher and player placement), overlaid with the cluster graph of §5.1 so that agent traffic is visible *on* the machines that carry it, next to the SLURM capacity badge (idle/busy/down). R7 reads directly off the picture: the same physical node hosts both an agent’s runtime and the keeper that captures it, and the operator sees both roles at once.

Fleet: the population at a glance (Fig. 5.4). One cell per node, colored by rolled-up health state (ok / warn / crit, idle and busy dimmed), worst first; the header aggregates events, errors, and cost over the selected window. The representation change argued in §5.3 pays off here: where the node-link graph saturates at a dozen nodes, the heatmap stays legible at hundreds (the capture below shows 256), and the red cells are the Q4 answer before a single query is typed. Clicking a cell filters the fleet to that node and hands off to the other views for the why.

Traces: where the time went (Fig. 5.5). The critical-path surface of §5.4: traces listed slowest-first; the selected trace rendered as a depth-indented waterfall with the critical path outlined and each hop labeled *tool* → *host*; the header names the bottleneck hop with its self-time: in the capture, `call_remote_agent` → `comp-04` at 12.0s of a 21.6s trace. A self-time bar above the waterfall shows at a glance which host consumed the wall-clock, and clicking any span opens the edge’s detail. This is Q3 answered as a sentence, not a scroll.

Together the five views implement the pivot the case study of §6.7 times end-to-end: Fleet says *which machine*, Health says *what and why*, Traces says *where the time went*, Scenarios says *who*, and the conversation view says *show me*. All are projections of one shared log, reachable from each other in a click or two.

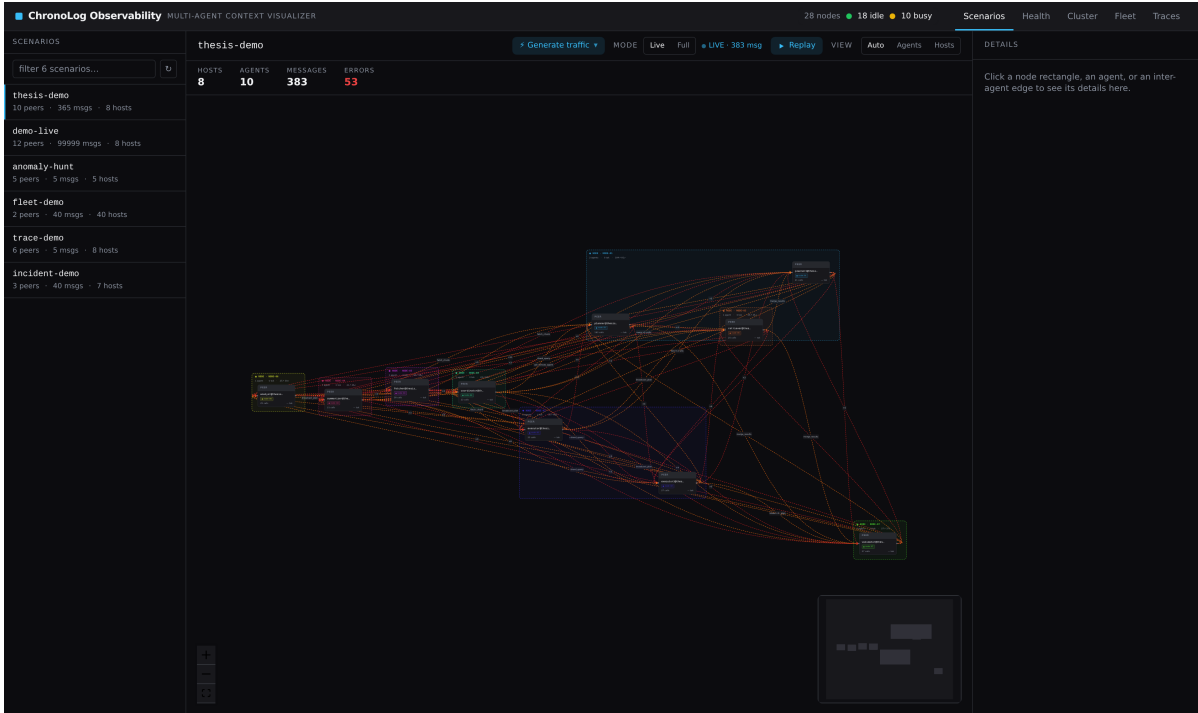


Figure 5.1: The Scenarios view, live: 10 agents across 8 host groups, every inter-agent call an edge (errors red), with live message and error counters. Clicking a node opens the agent’s conversation, the capture-to-conversation join of R1.

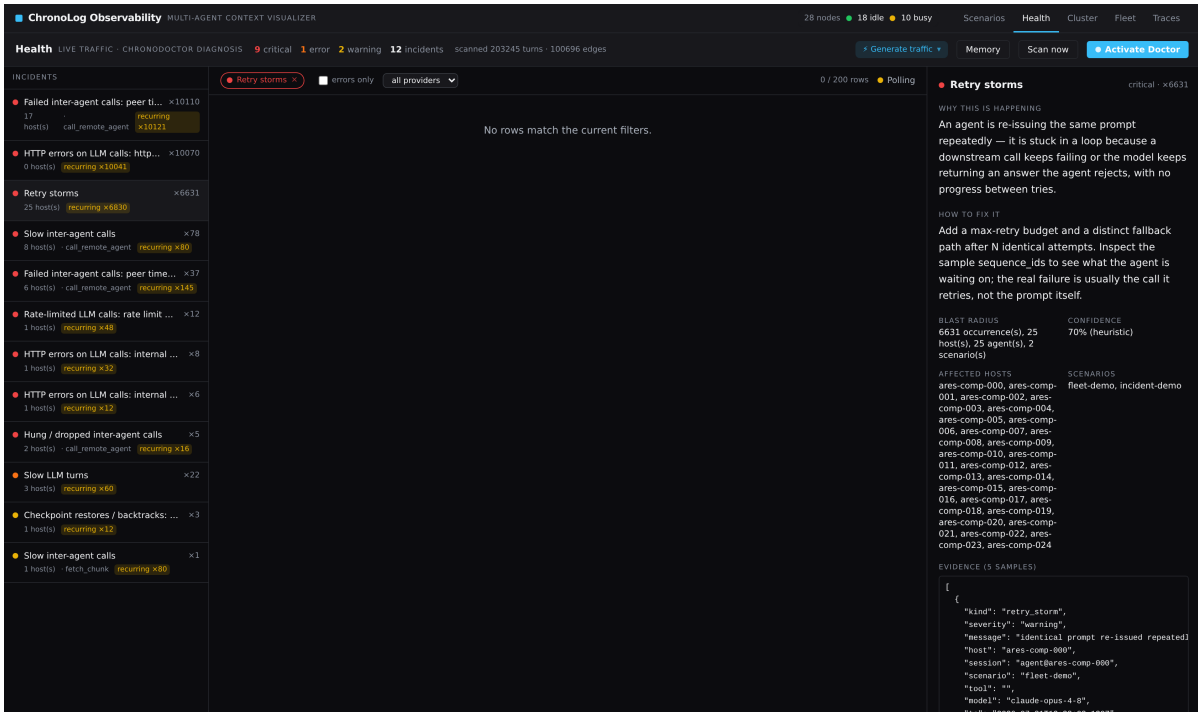


Figure 5.2: The Health view after a Doctor scan over a large demo corpus: 12 ranked incidents from ~200k scanned turns (left), and the selected incident’s diagnosis card (right) carrying root cause, remediation, blast radius, confidence, and evidence samples. *Recurring* badges are read from the cross-run incident memory.

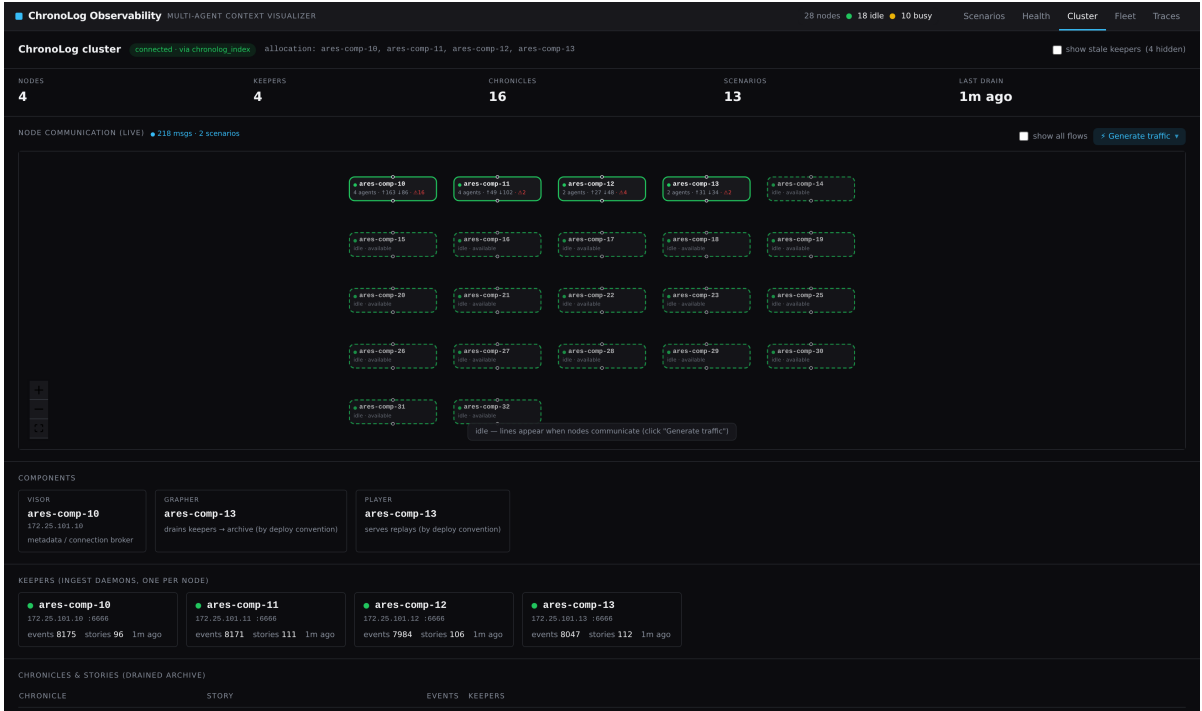


Figure 5.3: The Cluster view on a live 4-node deployment: ChronoLog components (visor, grapher, player placement; one keeper card per node with its event and story counts and last-drain age) alongside the live cluster graph and SLURM capacity. The headline freshness stat reports the *measured* last-drain age (§6.3); the grapher’s configured 180s acceptance window is relegated to its tooltip.

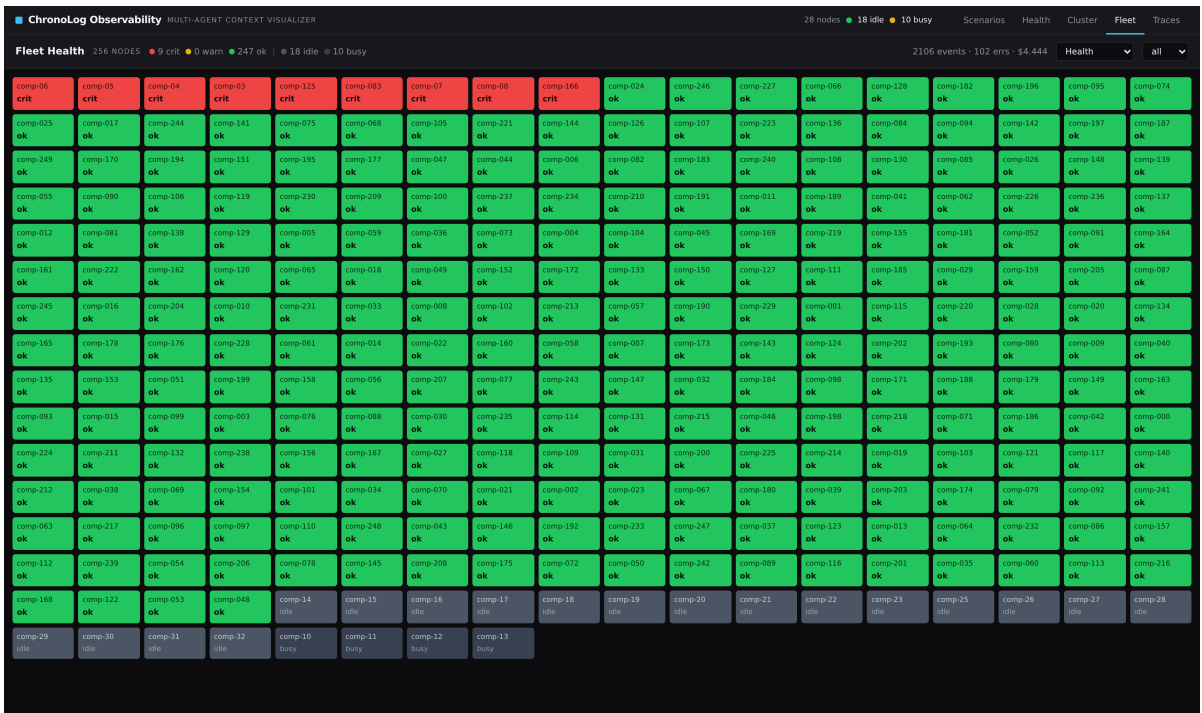


Figure 5.4: The Fleet view: 256 nodes, one cell each, rolled-up health worst-first. Nine critical nodes are the immediate Q4 answer. Rollup-backed reads keep this view interactive independent of event volume (§6.5).

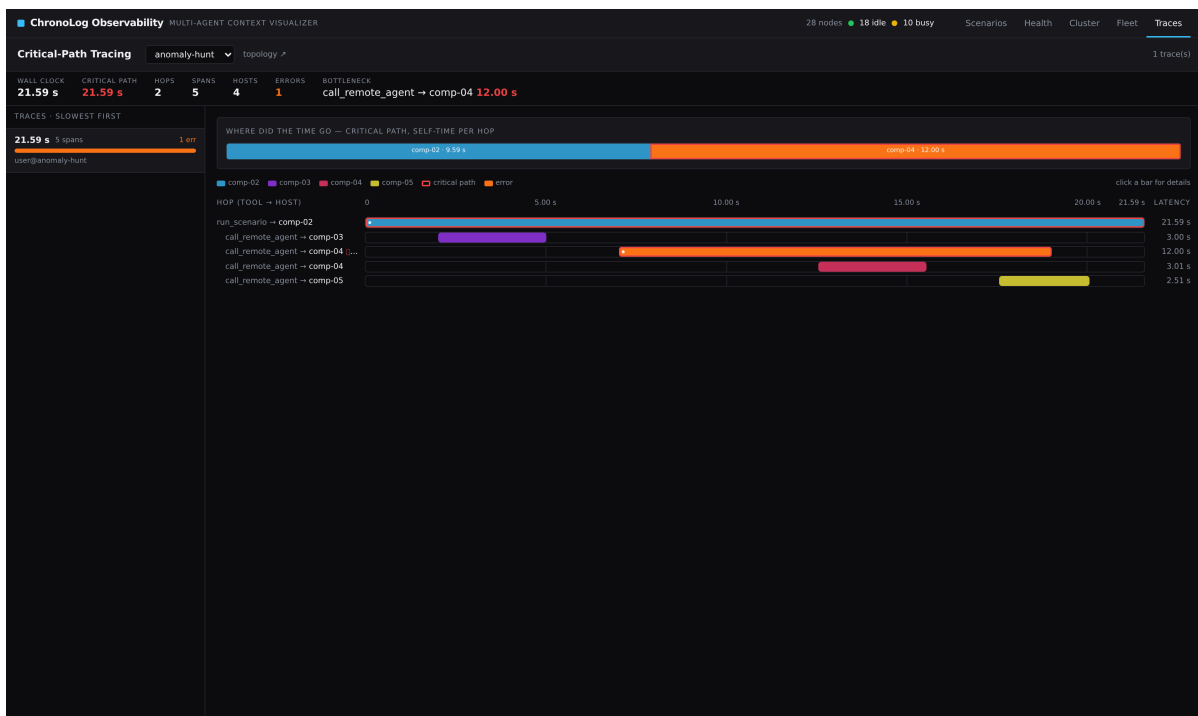


Figure 5.5: The Traces view: critical-path waterfall for the slowest trace of a scenario. The header names the bottleneck (the `comp-04` hop, 12.0s self-time of 21.6s wall-clock); the top bar shows self-time per hop along the critical path.

Chapter 6

Evaluation

The evaluation asks six questions, mapped to the requirements of §3.2: what does capture cost the observed workload (R5, §6.2); how fresh are the live views (R2, §6.3); is ChronoDoctor’s diagnosis *correct*, not merely cheap (R6, §6.4); do the analyses scale as claimed (R4, §6.5); does the whole stack work end-to-end on a real cluster with real agents (R1, R3, R7, §6.6); and does the system actually shorten the path from symptom to cause (§6.7). Section 6.8 discusses limitations. Table 6.1 previews the headline results.

6.1 Methodology

Platform. All live experiments run on the ARES cluster under SLURM. Each experiment allocates M idle compute nodes and deploys ChronoLog across them (visor, one keeper per node, grapher draining to an archive on shared storage), followed by the observability components: one collector and one capture-worker process per node, and one dashboard on a designated node. The deployments evaluated live use $M = 4$ and $M = 8$. In this configuration the keeper seals 10s story chunks under a 15s acceptance window, while the grapher and player run with a 180s acceptance window; §6.3 measures what these settings translate to end-to-end, and corrects a folklore number in the process.

Workloads. Two workloads exercise the identical ingest path. *Real agents*: Claude Agent SDK agents, one or more per node (configurable as K agents per node), each performing genuine LLM turns and one cross-node delegation through a remote-agent MCP tool per round; turns enter Path-A, edges Path-B. *Synthetic traffic*: a generator that emits realistic interaction records and **start/done** edge pairs through the same spool and collector endpoints, with configurable topology (mesh, star, pipeline, ring), agent count, node spread, error rate, and pacing. Real agents provide ground truth end-to-end; synthetic traffic provides controlled, reproducible, zero-token-cost load for overhead and scalability measurements, and is the fault-injection vehicle for §6.7.

Measurements. Latency overhead is measured as the difference in per-turn wall-clock between instrumented and uninstrumented runs of the same agent workload; capture-side costs by sampling the sidecar processes’ CPU and resident memory under load; storage by bytes per record in each tier; freshness by timestamping an event at emission and at receipt in a subscribed SSE client. Each is reported with distribution statistics over repeated runs.

Question	Measured answer
What does capture cost the agent?	3.6 ms per turn-plus-delegation, < 0.05% of a mean turn; the A/B delta’s confidence interval spans zero (§6.2).
How fresh are live views?	10.7 ms emit→operator screen, ~1 900× inside the measured 20–30 s durable drain (§6.3).
Is diagnosis correct?	580/580 planted faults detected, 0 false positives on clean background, 500 raw signals → 1 ranked card (§6.4).
Do the analyses scale?	Fleet reads flat at 19–24 ms from 10 k to 500 k events (457× over raw folding); heatmap 36 ms at 250 nodes (§6.5).
Does it run for real?	28/28 end-to-end checks on live 4- and 8-node clusters; up to 32 real agents with flat per-node sidecar load (§6.6, §6.5).
Does it shorten diagnosis?	First-time operator: complete verified diagnosis in 72 s via the dashboard vs. 0/3 questions answered from raw logs (§6.7).

Table 6.1: The evaluation at a glance. Every number is measured; the sections referenced give conditions, distributions, and caveats.

6.2 Capture Overhead

Capture overhead is the cost the observed workload pays, and the design predicts it is small by construction: the Path-A fast path is one local JSONL append (no network, no lock, no downstream dependency, §4.3); the Path-B fast path is one HTTP POST to a daemon on `localhost` that enqueues and returns. Both are constant-time and independent of history size. Everything else (the ChronoLog write, the forward, the drain) happens in sidecar processes off the agent’s critical path.

Table 6.2 reports the measured costs. All latency numbers are taken on live compute nodes with the production NFS state directory: the real deployment condition, not a tmpfs microbenchmark.

Cost	Path	Metric	Measured
Spool append	A	added latency per turn (p50 / p95)	0.83 / 1.09 ms
Collector POST	B	added latency per call (p50 / p95)	2.82 / 6.22 ms
fallback (collector down)	B	direct POST to dashboard (p50 / p95)	8.95 / 12.75 ms
End-to-end turn delta	A+B	instrumented vs. not, per turn	+365 ± 2 398 ms
Collector daemon	B	CPU / RSS per node, 20 ev/s	32% / 40 MB
Capture worker	A	CPU / RSS per node, same load	0.8% / 51 MB
Interaction record	A	bytes per event (spool / archive)	2 093 / 2 173 B
Edge event	B	bytes per event (hot / archive)	612 / ≈560 B
Live forward	B	bytes per event on the wire	646 B

Table 6.2: Capture overhead, measured on a live 8-node deployment (compute nodes, NFS state directory; $n=10\,000$ per latency row). The headline row is the end-to-end A/B turn delta over real LLM turns: its 95% confidence interval spans zero: capture is statistically invisible inside the workload’s own variance. Capture worker CPU is the mean of a bursty profile (idle between 5 s drain passes, peaks of 85% during them); the wire number is amortized over batches of ≤ 64 (658 B unbatched).

The headline is easy to over- and under-claim, so it needs stating carefully. The A/B experiment ran $n=30+30$ real agent turns (identical prompts, interleaved to absorb provider drift): instrumented turns averaged $8\,099 \pm 5\,147$ ms, uninstrumented $7\,734 \pm 4\,291$ ms, a delta of +365 ms whose 95% confidence interval ($\pm 2\,398$ ms, Welch $t = 0.30$) makes it statistically indistinguishable from zero. The claim therefore rests on both measurements: the *mechanism*

costs $0.83 + 2.82 \approx 3.6$ ms per turn-plus-delegation (under **0.05%** of a 7.7 s mean turn), and the A/B run confirms that nothing beyond the mechanism leaks onto the critical path. With the collector *down*, the fallback POST to the dashboard costs $3\times$ the normal path (8.95 vs. 2.82 ms p50): degraded mode is slower, not cheaper, which is why the collector exists.

Two structural observations frame the numbers. First, the denominator is large: an LLM turn takes seconds and a delegation round-trip longer, so capture cost in milliseconds translates to a per-turn overhead orders of magnitude below the workload’s own variance. This is the inverse of the classical tracing regime (§2.2.3), where sub-millisecond RPCs made even microseconds of instrumentation significant; it is why complete capture is affordable here. Second, the failure-mode cost is bounded by design rather than measured luck: if every downstream component dies, the agent’s cost is unchanged (Path-A) or one failed local connection followed by a fallback POST (Path-B).

6.3 Freshness and Query Latency

Freshness is the interval from an agent emitting an event to that event being visible to an operator. The claim under test is that the hot path (R2) puts the event on screen in milliseconds while the cold path delivers durability in tens of seconds, and the measurement corrects a number this thesis itself repeated from deployment folklore.

Table 6.3 reports the measured hot-path pipeline (emit $\rightarrow$ collector $\rightarrow$ dashboard ingest $\rightarrow$ SSE frame at a subscribed client), the measured cold-path drain, and per-view query latencies against the populated live deployment; Figure 6.1 plots the spread.

Measure		Measured
Emit $\rightarrow$ SSE frame at client	p50 / p95	10.7 / 11.8 ms
direct-ingest fallback	p50 / p95	8.5 / 8.9 ms
Cold-path availability (drain)	p50 / max, $n=10$	20 / 30 s
Scenario graph query	p50	470 ms
Cluster graph query	p50	2 009 ms
Fleet health query (raw fold)	p50	1 881 ms
rollup-backed (§6.5)	p50	19–24 ms
Doctor scan (bounded gather)	wall time	1 807 ms
Trace reconstruction	per scenario	282 ms

Table 6.3: Freshness and query latency on the live 8-node deployment (dashboard node, $n=100$ per query row, $n=1\,000$ paced events at 20 ev/s with zero loss for the SSE rows; corpus ≈ 22 k archived events). The hot path sits three orders of magnitude inside the measured drain: an event is on the operator’s screen $\sim 1\,900\times$ sooner than it is readable from the archive (p50 drain over p50 push).

The drain is 20–30 s; the “ ~ 180 s” was something else. Measured directly (append a marked event, poll the archive), a story chunk becomes readable **20–30 s** after append ($n=10$, p50 20 s), governed by the keeper’s 10 s chunking and 15 s acceptance window plus the grapher’s write batch, not by the 180 s window configured downstream. The ~ 180 s that operators (and earlier drafts of this thesis) attributed to the drain is in fact the client’s *replay-query timeout*. In these deployments the player’s playback responses never reach the dashboard process’s receiver service: the player logs every request and completes none of the transfers toward that endpoint, while an otherwise-identical receiver in the capture process works. Every live replay issued by the dashboard therefore blocks for the full `CL_ERR_QUERY_TIMED_OUT` window of ~ 180 s: per story, per request, holding the client lock. Diagnosing this (§6.8) explained the folklore number and motivated generalizing the archive-first read rule of §4.4 from listing indexes to *all*

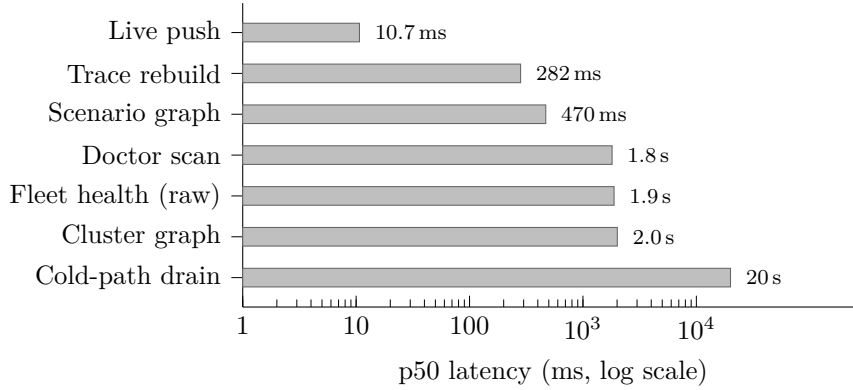


Figure 6.1: The freshness and query-latency spread on one log axis (p50, live 8-node deployment). Live push takes single-digit milliseconds and scenario-scoped reads stay sub-second, while population-wide reads that fold the raw archive sit near 2s on this corpus. That is the linear-in-events cost the rollout design of §6.5 flattens to roughly 20 ms.

dashboard replays: with the drain at 20–30 s, serving full-fidelity views from the archive costs at most half a minute of staleness and removes the last blocking read from the serving process.

Qualitatively, the split then behaves as designed in every live run: the live views (scenarios, cluster, fleet, health, traces) render immediately during a run, while full-fidelity conversation views fill in within the half-minute drain; no view blocks the dashboard, fresh clusters included.

6.4 Detection Quality

R6 demands that diagnosis be free of marginal token cost; this section asks whether the deterministic pipeline that satisfies it is also *correct*. The experiment scores a full ChronoDoctor scan (gather, detect, cluster, diagnose, remember; §5.2) against a ground-truth manifest of injected faults. A controlled corpus is seeded offline through the same spool and inter-agent-store write paths the capture layer uses: a clean background of healthy traffic (216 turns, 120 completed delegations), plus ten planted faults of each kind the detectors claim to cover, plus one *storm* (500 identical peer-timeout edges spread across 40 hosts) to test the compression claim of §5.2. Each planted fault is shaped to carry exactly one symptom (single-label ground truth), so every detection is unambiguously attributable. The same fault mix is then injected a second time and the corpus re-scanned, testing whether the incident memory recognizes the recurrence. Table 6.4 reports the result.

Three of the numbers carry the argument. First, the clean background produces *zero* incidents: the detectors’ thresholds (medians with absolute floors, consecutive-repeat minima) do not fire on healthy variance, so a ranked incident list starts empty rather than requiring the operator to learn a noise floor. Second, compression behaves as designed: 580 injected raw signals become 9 ranked incident cards; in particular the 500 identical timeouts collapse into *one* card whose count, host list, and severity escalation carry the blast radius (Q2’s “not a scroll of per-event errors”). Third, the longitudinal claim (Q5) holds mechanically: after the first scan writes the incident memory, the second scan badges all nine incidents as recurring with the correct prior count, read back from the self-compacting memory document rather than from process state.

The scope of the claim is deliberately bounded. The planted faults are in-distribution and comfortably above the detector thresholds: the experiment establishes that detection, clustering, diagnosis routing, and cross-run memory are correct on unambiguous faults, not that the thresholds are well-calibrated on borderline cases. “Diagnosis correct” likewise means

Fault kind	Inj.	Det.	FP	Cards	Diagnosis	Recurring
Interaction 5xx errors	10	10	0	1	yes (0.60)	yes
Rate-limited calls (429)	10	10	0	1	yes (0.85)	yes
Retry storms	10	10	0	1	yes (0.70)	yes
Latency outliers	10	10	0	1	yes (0.55)	yes
Edge errors (peer timeout)	10	10	0	1	yes (0.70)	yes
Orphan calls (no done)	10	10	0	1	yes (0.70)	yes
Slow edges	10	10	0	1	yes (0.55)	yes
Recovery events	10	10	0	1	yes (0.60)	yes
Storm: 500× identical timeouts	500	500	0	1	yes (0.70)	yes
Clean background	0	—	0	0	—	—

Table 6.4: ChronoDoctor detection quality against a ground-truth fault manifest (offline, seeded through the production write paths; `e6_doctor_quality.py` in `scripts/eval/`, seed-deterministic; results identical at $n=25$ and under a different seed). “Cards” is the number of incident cards produced per fault kind; “Diagnosis” marks whether the heuristic diagnosis names the planted root-cause category, with the engine’s stated confidence; “Recurring” marks whether the second scan badged the incident with its prior count.

only that the incident reached the intended rule of the hand-built rule set, whose coverage limits are discussed in §6.8. A perfect score under these conditions is the expected behavior of deterministic detectors, not evidence against harder inputs; the informative results are the zero-false-positive background, the exact $500 \rightarrow 1$ compression, and the memory round-trip.

6.5 Scalability

The evaluation tests three scaling claims.

Fleet reads are independent of event volume. The rollup design (§5.3) makes the fleet query $O(\text{nodes} \times \text{buckets})$; the raw-fold fallback is $O(\text{events})$. The experiment holds nodes and window fixed (16 nodes, 60-minute window) and grows the event count from 10^4 to 5×10^5 , comparing the fleet-health query latency of the two paths (20 requests per point, p50). Figure 6.2 shows the result: rollup-backed reads stay flat at 19–24 ms across a $50\times$ growth in event volume, while the raw fold grows linearly (slope ≈ 17 ms per thousand events between the endpoints): 107 ms at 10 k events, 8.5 s at 500 k, a $457\times$ gap at the largest size. The offline fleet harness, which seeds hundreds of nodes’ worth of bucketized history, summarizes a 100–250-node fleet in interactive time (15 ms p50 at 100 nodes, 36 ms at 250), where the same query folded from raw events would touch every record ever captured.

Representation, not just computation, must scale. The node-link cluster graph is legible to roughly a dozen nodes and degenerates beyond; this is a limit of the visual form, not of the data. The fleet heatmap holds one cell per node and remains legible at hundreds; its rendering cost grows with nodes only. This is why the system changes representation between Q1 (topology, small M , maximal detail) and Q4 (population, large M , one health value per node).

Agents per node. The real-agent harness scales in two dimensions: nodes (M) and agents per node (K). Capture is per-node (one collector, one keeper, one worker regardless of K), so per-node capture load grows linearly in K with no cross-node coordination, and cross-node load grows only with actual inter-agent traffic. Live runs swept $K \in \{1, 2, 4\}$ at $M = 8$ with real agents (every agent performing genuine LLM turns and one cross-node delegation): all 8,

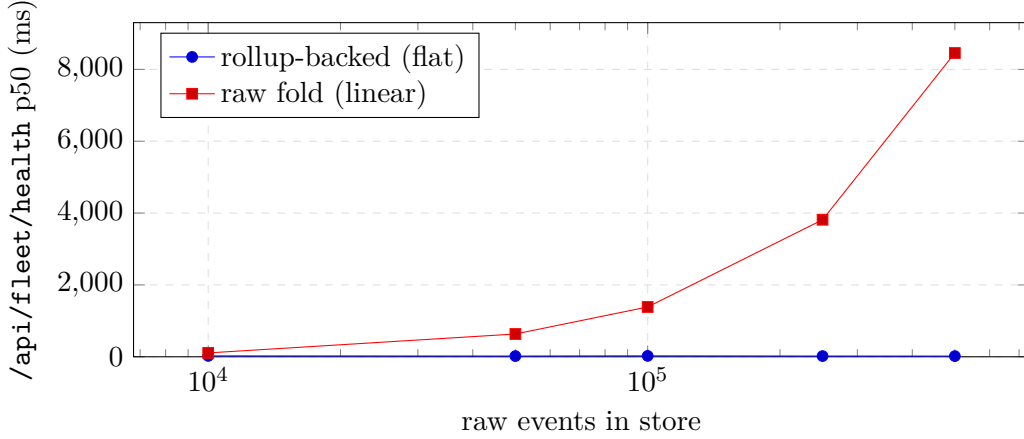


Figure 6.2: Fleet-health query latency vs. stored event count (16 nodes, 60-minute window, p50 over 20 requests). Pre-aggregated rollup reads are independent of event volume; the raw-fold fallback is linear in it. Measured offline via `bench_rollup_scaling.py` in `scripts/eval/`.

16, and 32 agents exited cleanly and every delegation round-tripped through the archive (16, 32, and 64 edge records respectively). The per-node sidecar was flat across the sweep: the collector held 0.1–0.3% mean CPU (transient forward bursts to ~21%) and 37–39 MB RSS at every K , and the capture worker 0.3% mean at 48 MB. These means sit far below Table 6.2’s 32% figure because the regimes differ: the table’s collector serves a sustained 20 ev/s synthetic stress load, while real agents (one turn every several seconds) offer a small fraction of that rate. Both observations are consistent with the claim that agent count stresses the LLM provider, not the capture plane. The largest configuration exercised cleanly end-to-end is therefore $K \times M = 4 \times 8 = 32$ real agents.

6.6 Functional Validation

The system’s end-to-end correctness claim, capture through substrate to every view with real agents on a real cluster (R1, R3, R7), is validated by an automated harness rather than by demonstration. The harness brings up the full deployment on a live allocation, runs real Claude agents (genuine LLM turns; one cross-node MCP delegation per agent), waits out the drain, restarts the dashboard cold against the same substrate (so everything shown is reconstructed from durable state, not process memory), and then asserts machine-checkable properties of every API the dashboard serves: each view is populated with the expected entities, per-agent metrics are non-zero where turns occurred, and inter-agent edges exist with correct endpoints. The strictest check is the cross-plane join: the scenario graph’s clicked node resolves to that agent’s conversation with its turns and token counts, exercising the session-identity contract of §4.2 end to end.

The harness passes **28/28** checks on live 4-node and 8-node ChronoLog clusters. Beyond the pass count, the harness is the regression net that hardened the design: the pure-reader rule, the archive-first index read, and the hostname canonicalization of §4.5 were each introduced to turn a live-cluster failure of this harness into a pass, and remain pinned by it.

6.7 Case Study: Fault Injection and Localization

The final question is whether the pieces compose into the cross-layer diagnosis the thesis promises. The scenario: on a live M -node deployment carrying steady background traffic,

the synthetic generator injects a retry/timeout storm concentrated on one node, alongside a delegation leg that starts and never completes.

The observed localization path:

1. **Detect and compress (Q2).** A Doctor scan returns not hundreds of error rows but a short ranked incident list: one incident aggregating the storm (count in the hundreds, blast radius naming the affected hosts and sessions, rate-limit/timeout root cause and remediation attached), and one *critical* orphan-call incident for the hung delegation, a failure invisible in any per-edge view, since its signature is an absence (§5.2).
2. **Localize to the machine (Q4).** The fleet heatmap shows the population: one cell red among ok nodes (the storm’s node), turning “the fleet looks unhealthy” into “it is this machine.”
3. **Attribute the latency (Q3).** The trace waterfall for the affected scenario highlights the critical path; the bottleneck hop, charged by self-time, lands on the same node.
4. **Reach the culprit (Q1).** Clicking the offending agent in the scenario graph opens its conversation (the actual turns, retries and costs of the misbehaving agent), completing the pivot from fleet symptom to a single conversation in two clicks.
5. **Remember (Q5).** Re-running the same fault and re-scanning badges the incident **recurring** $\times N$: the second occurrence is recognized, not rediscovered.

Timed comparison. The path above was then measured as a controlled exercise on the live 8-node deployment. Faults were injected by script (a 6-turn identical-prompt 5xx retry storm on one node, twelve peer-timeout delegations into it, and one delegation leg killed after its **start** frame), and a diagnostician answered three questions, once through the dashboard and once from the raw capture files with shell tools only, against a verifiable ground truth: *which node, and what error* (CS1); *which session is retrying, on what prompt* (CS2); *which delegation never completed, by caller→callee and correlation id* (CS3; labeled CS to avoid collision with the operator questions Q1–Q5). The operator was the author, timing each question from first interaction to stated answer, with every answer checked against the injected values (including the orphan’s correlation id). Table 6.5 reports the result.

Operator	Interface	CS1	CS2	CS3	Outcome
Human, first exposure	dashboard	35 s	17 s	20 s	3/3 correct (72 s)
Human, second pass	dashboard	10 s	5 s	5 s	3/3 correct (20 s)
Human, third pass	dashboard	5 s	5 s	5 s	3/3 correct (15 s)
Human	raw logs	abandoned at ~5–10 min			0/3
LLM agent (supplementary)	raw logs	44 s wall-clock, 4 commands			3/3 correct

Table 6.5: Timed diagnosis of the injected fault, $n=1$ operator (first-ever exposure to the task). Dashboard trials repeat the same questions, so passes 2–3 measure a practiced operator. The raw-log attempt ran *after* the dashboard trials (answer leakage, if any, favored the raw-log path) and still produced no answers: the operator could not establish a starting point in the JSONL directories. The supplementary row is a fresh LLM agent given only the three questions and the data directories (no ground truth, shell text tools only); machine wall-clock is not comparable to human minutes and is reported for what it demonstrates about the *data*, not the operator.

Two findings, one expected and one not (the second previewed in §2.6.3). The expected one: a first-time operator produced a complete, verified diagnosis through the dashboard in **72 seconds**, yet could not answer *any* of the three questions from the raw files, abandoning

after five to ten minutes without a foothold. The dashboard run included recovering from one wrong pick on CS3: the twelve errored-but-completed delegations rendered adjacent to the true orphan, and the correlation-id check caught the confusion. The unexpected one: the LLM-agent baseline answered all three questions from the raw JSONL in 44 seconds and four shell commands, finding the orphan by counting correlation-id occurrences, the very question the protocol predicted would time out under `grep`. Together they sharpen what this chapter can claim: the capture layer’s raw data is *sufficient* for complete diagnosis, but it is not *accessible*: extracting the answer requires either expertise the on-call operator does not have or tooling that encodes it. The dashboard is that encoding: it substitutes for operator expertise, not for data.

The caveats are plain: one operator, one injected incident on a small scenario corpus, and a UI defect surfaced by the CS3 stumble (never-completed delegations rendered indistinguishably from completed ones). That defect was fixed following the study. Open delegations now carry a dedicated visual class (amber, dashed, badged *never completed*) across the scenario graph, the edge detail panel, and the trace waterfall, and hung flows outrank errors in edge selection; the pathology that cost the operator their one wrong pick is now the most visually distinct element on screen.

6.8 Discussion and Limitations

Hot store locality. The hot store and live bus live with the dashboard process; they are not replicated. A dashboard failure loses no data (the log is authoritative; the hot view rebuilds from backlog and, after the drain, from ChronoLog) but interrupts live observation: acceptable for an operator tool, but a real asymmetry with the replicated cold path.

The drain window is inherited; the replay path is broken beneath us. The measured 20–30s cold-path latency is ChronoLog’s configuration, not this system’s contribution; the design works around it rather than removing it, and a substrate with tighter chunking would narrow the hot/cold gap without changing the architecture. A harder inherited fact surfaced during the evaluation itself: in these deployments the player’s playback responses are never delivered to the dashboard process’s receiver service (requests arrive in the player’s log; no transfer toward that endpoint ever completes; an otherwise-identical receiver in the capture process works), so every live replay from the dashboard burns the ~ 180 s query timeout while holding the client lock. The transport-level cause remains undiagnosed; it sits inside the ChronoLog client/player pair, below this system’s code. The response was to generalize the archive-first rule: with `CHRONOLOG_ARCHIVE_FIRST` set, the dashboard and capture worker serve all replay reads from the grapher’s archive and never ask the player, bounding worst-case staleness at the measured drain window. This is the correct engineering trade under the observed substrate, but it means the live-replay path is effectively unused, and a substrate fix would recover it.

Diagnosis coverage. The heuristic diagnoser is first-match-wins over a hand-built rule set: specific on the failure classes it encodes, and explicitly low-confidence outside them. It explains known pathologies; it does not discover novel ones. The signature scheme can also split one logical failure into two incidents if its message text varies beyond what normalization strips.

Evaluation scale. Live validation ran at $M \in \{4, 8\}$ with real agents (the allocation sizes available), while the 100+-node claims rest on the rollup complexity argument and seeded fleet-scale data rather than a live fleet of that size. The p95 fold of §5.3 is upper-biased by construction, and inferred (parentless) traces rely on time nesting, which can mis-parent

overlapping concurrent delegations between the same pair of agents; explicit parent propagation removes the ambiguity where the instrumentation provides it.

Taken together, the six questions this chapter opened with have quantitative answers: capture is statistically invisible to the agent, live views run three orders of magnitude ahead of the durable drain, detection is exact on ground truth, the fleet reads scale as designed, the stack survives real agents on real clusters, and the path from symptom to cause takes an untrained operator seventy-two seconds. What these answers add up to, and what they cost, is the subject of the concluding chapter.

Chapter 7

Conclusions and Future Work

This chapter closes the thesis: it checks the outcome against the requirements of Chapter 3, states the design lessons the work argues for, and lays out the directions it opens.

7.1 Conclusions

This thesis set out from a one-sentence gap: the tools we surveyed answer “what did my agent do?” with a span tree of one run on one machine, and, to our knowledge, none answers “what is my agent *fleet* doing across the cluster, and which node is dragging it down?” The claim defended across the preceding chapters is that the gap closes naturally once the data is stored in its native shape: a multi-agent execution is a causally-ordered event log, and a distributed shared log with a capture process on every node (ChronoLog) makes the coupling of agent semantics to physical topology a property of the substrate rather than an integration problem.

Checking the outcome against the requirements of §3.2, each now with its measured answer from Chapter 6: **R1** (dual-plane causal capture) is met by the two capture paths and the `role@scenario` identity contract, under which every event carries its host and the planes join by construction, verified end-to-end by the click-through check of the live harness. **R2** (real-time visibility) and **R3** (durability with exactly-once ingest) are met jointly by the hot/cold split: an event reaches the operator’s screen 10.7 ms after emission and the durable archive 20–30 s after (the hot path three orders of magnitude inside the cold), with at-least-once spooled capture deduplicated by high-water marks rebuilt from the log itself. **R4** (fleet scale) required changing both computation and representation: per-(host, minute) rollups hold the fleet read at 19–24 ms across a $50\times$ growth in event volume where the raw fold grows to $457\times$ slower, and the heatmap answers at 250 nodes in 36 ms where node-link rendering fails at a dozen.

R5 (minimal overhead) holds structurally and is confirmed empirically: the agent-side mechanism costs 3.6 ms per turn-plus-delegation, under 0.05% of a mean turn, and the end-to-end A/B delta over real LLM turns is statistically indistinguishable from zero. **R6** (zero-cost diagnosis) rests on the deterministic detect–cluster–diagnose pipeline with its self-compacting cross-run memory, LLM assistance strictly opt-in; ground truth confirms the pipeline correct: 580/580 planted faults detected with zero false positives, a 500-signal storm compressed to one ranked card, and recurrence recognized across runs. **R7** (drop-in deployment) is demonstrated by the live harness, which attaches the full stack to a plain SLURM allocation without modifying ChronoLog, the scheduler, or the agent SDK: 28/28 automated checks on 4- and 8-node clusters, sustained at 32 concurrent real agents with flat per-node sidecar load.

Beyond the artifact, the thesis argues two transferable design lessons. The first is architectural. Observability systems are conventionally assembled as instrumentation plus a bespoke backend; for workloads whose events are few, expensive, and causally structured (as LLM-agent fleets are), inverting the architecture around a general distributed shared log yields the cross-layer joins, the durability, and the replayability for free, at the price of engineering around

the log’s read behavior. That price is real but modest, and the evaluation quantified both of its parts: the planned part (the hot/cold split of Chapter 4) and the unplanned part (§6.8’s broken replay path, absorbed by generalizing archive-first reads at a bounded cost of half a minute of staleness). What it buys is the capability neither surveyed camp offers: the pivot from a fleet-wide symptom to a single agent’s conversation on a named machine. The case study measured that pivot at 72 seconds, questions to verified answers, for an operator who had never seen the task before.

The second lesson comes from the same case study’s baseline, and it reframes what an observability *interface* is for. The raw captured log proved *sufficient* for complete diagnosis (an LLM agent given only the data directories answered all three questions, orphaned delegation included, in under a minute), while the human operator, given the same files, could not begin. The dashboard’s contribution is therefore not access to data but *encoded expertise*: each view is a frozen form of the queries an expert would write. Systems built on a sufficient substrate can choose where that expertise lives (in views for humans, or in agents that query the log directly), and the shared-log architecture serves both consumers from the same store.

7.2 Future Work

Runtime causal queries. The substrate stores the full causal event history, but the analyses of Chapter 5 are pre-built views. The next step is the Pivot-Tracing move [20]: ad-hoc queries evaluated along causal edges (“the total token cost of everything that transitively touched agent *B*”), which the log’s ordering and the correlation schema already make expressible.

Replicated hot path. Promoting the hot store and live bus from per-dashboard-process structures to a small replicated service would remove the single point of live observation and allow multiple dashboards over one fleet.

Recovering live replay. The archive-first mode of §6.8 is the correct trade under the observed substrate, but it leaves ChronoLog’s player unused and pins full-fidelity staleness to the drain window. Diagnosing the transport-level fault beneath it (playback responses that never reach one process’s receiver while reaching another’s on the same host) is upstream work on the ChronoLog client/player pair; fixing it would let the dashboard serve un-drained history live and shrink the only staleness the design still carries.

Scale, sampling, and adaptivity. Complete capture is the right default at today’s event rates; at orders of magnitude more agents, selective capture of payload content (while always keeping the causal skeleton) would be the principled degradation. Live validation at hundreds of real nodes, closing the gap that §6.8 concedes, is chiefly an allocation problem.

Richer diagnosis. The heuristic diagnoser’s rule set could be learned rather than hand-built (mining the incident memory across runs for recurring signature–remediation pairs), and the opt-in LLM diagnoser could be grounded on retrieved evidence (traces, fleet state) rather than the incident summary alone.

Agent-facing observability. The case-study baseline, an LLM agent diagnosing the fleet from the raw log in under a minute, is less a curiosity than a signpost: the next consumer of observability data may itself be an agent. The captured log already has everything such a consumer needs (causal order, correlation ids, host attribution, JSONL it can read with four shell commands); the obvious extension is a first-class query surface, an MCP tool over the log and its analyses, so that a supervising agent can run the Doctor’s gather-and-detect loop, or

answer ad-hoc causal questions, as part of the fleet it watches. The zero-marginal-cost principle of R6 would then bound how much of that supervision runs on heuristics versus paid model calls.

Interoperability. Emitting and ingesting the OpenTelemetry generative-AI conventions [23] would let the system exchange data with the Camp-1 ecosystem, their span semantics mapped onto the log’s event schema, so that adopting the fleet layer does not mean abandoning existing per-agent tooling.

The direction of the field is not in doubt: agent systems are growing in both population and physical extent, and the questions operators ask are increasingly the questions distributed systems have always asked, now with a conversation attached. This thesis’s contribution is to show that the sixty-year-old toolbox of that discipline, from happened-before to the shared log, is not merely applicable to the newest workload but is its natural home.

Bibliography

- [1] Anthony Agelastos, Benjamin Allan, Jim Brandt, Paul Cassella, Jeremy Enos, Joshi Fullop, Ann Gentile, Steve Monk, Nichamon Naksinehaboon, Jeff Ogden, Mahesh Rajan, Michael Showerman, Joel Stevenson, Narate Taerat, and Tom Tucker. The lightweight distributed metric service: A scalable infrastructure for continuous monitoring of large scale computing systems and applications. In *SC '14: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 154–165, 2014.
- [2] AgentOps. Agentops: Observability for AI agents. <https://www.agentops.ai>, 2026. Accessed July 2026.
- [3] Anthropic. Model context protocol. <https://modelcontextprotocol.io>, 2024. Specification; accessed July 2026.
- [4] Arize AI. Phoenix: Open-source AI observability and evaluation. <https://phoenix.arize.com>, 2026. Accessed July 2026.
- [5] Mahesh Balakrishnan, Dahlia Malkhi, Vijayan Prabhakaran, Ted Wobber, Michael Wei, and John D. Davis. CORFU: A shared log design for flash clusters. In *9th USENIX Symposium on Networked Systems Design and Implementation (NSDI 12)*, pages 1–14, 2012.
- [6] Nathan Bronson, Abutalib Aghayev, Aleksey Charapko, and Timothy Zhu. Metastable failures in distributed systems. In *Proceedings of the Workshop on Hot Topics in Operating Systems (HotOS)*, pages 221–227, 2021.
- [7] Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. Why do multi-agent LLM systems fail? arXiv preprint arXiv:2503.13657, 2025.
- [8] Michael Chow, David Meisner, Jason Flinn, Daniel Peek, and Thomas F. Wenisch. The mystery machine: End-to-end performance analysis of large-scale internet services. In *11th USENIX Symposium on Operating Systems Design and Implementation (OSDI 14)*, pages 217–231, 2014.
- [9] Datadog. LLM observability. <https://www.datadoghq.com/product/llm-observability/>, 2026. Accessed July 2026.
- [10] Colin J. Fidge. Timestamps in message-passing systems that preserve the partial ordering. In *Proceedings of the 11th Australian Computer Science Conference (ACSC)*, pages 56–66, 1988.
- [11] Haryadi S. Gunawi, Riza O. Suminto, Russell Sears, Casey Golliher, Swaminathan Sundararaman, Xing Lin, Tim Emami, Weiguang Sheng, Nematollah Bidokhti, Caitie McCaffrey, et al. Fail-slow at scale: Evidence of hardware performance faults in large production

- systems. In *16th USENIX Conference on File and Storage Technologies (FAST 18)*, pages 1–14, 2018.
- [12] Jonathan Kaldor, Jonathan Mace, Michał Bejda, Edison Gao, Wiktor Kuropatwa, Joe O’Neill, Kian Win Ong, Bill Schaller, Pingjia Shan, Brendan Viscomi, Vinod Venkataraman, Kaushik Veeraraghavan, and Yee Jiun Song. Canopy: An end-to-end performance tracing and analysis system. In *Proceedings of the 26th Symposium on Operating Systems Principles (SOSP 17)*, pages 34–50, 2017.
 - [13] Martin Kleppmann. *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O’Reilly Media, 2017.
 - [14] Anthony Kougkas, Hariharan Devarajan, Keith Bateman, Jaime Cernuda, Neeraj Rajesh, and Xian-He Sun. ChronoLog: A distributed shared tiered log store with time-based data ordering. In *Proceedings of the 36th International Conference on Massive Storage Systems and Technology (MSST 20)*, 2020.
 - [15] Jay Kreps. The log: What every software engineer should know about real-time data’s unifying abstraction. LinkedIn Engineering Blog, <https://engineering.linkedin.com/distributed-systems/log-what-every-software-engineer-should-know-about-real-time-datas-unifying>, 2013.
 - [16] Jay Kreps, Neha Narkhede, and Jun Rao. Kafka: A distributed messaging system for log processing. In *Proceedings of the 6th International Workshop on Networking Meets Databases (NetDB)*, 2011.
 - [17] Leslie Lamport. Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM*, 21(7):558–565, 1978.
 - [18] LangChain, Inc. Langsmith: Observability and evaluation for LLM applications. <https://smith.langchain.com>, 2026. Accessed July 2026.
 - [19] Langfuse. Langfuse: Open-source LLM engineering platform. <https://langfuse.com>, 2026. Accessed July 2026.
 - [20] Jonathan Mace, Ryan Roelke, and Rodrigo Fonseca. Pivot tracing: Dynamic causal monitoring for distributed systems. In *Proceedings of the 25th Symposium on Operating Systems Principles (SOSP 15)*, pages 378–393, 2015.
 - [21] Nathan Marz and James Warren. *Big Data: Principles and Best Practices of Scalable Realtime Data Systems*. Manning Publications, 2015.
 - [22] Friedemann Mattern. Virtual time and global states of distributed systems. In *Parallel and Distributed Algorithms*, pages 215–226. North-Holland, 1989.
 - [23] OpenTelemetry. Semantic conventions for generative AI systems. <https://opentelemetry.io/docs/specs/semconv/gen-ai/>, 2026. Accessed July 2026.
 - [24] Thang Duc Pham, Harikrishna Tummalapalli, Fakhrul Hasan Bhuiyan, Álvaro Vázquez Mayagoitia, Christine Simpson, Riccardo Balin, Venkatram Vishwanath, and Murat Keçeli. Multi-agent orchestration for high-throughput materials screening on a leadership-class system. arXiv preprint arXiv:2604.07681, 2026.
 - [25] Pixie Authors. Pixie: Instant kubernetes-native application observability. <https://px.dev>, 2026. CNCF sandbox project; accessed July 2026.

- [26] Prometheus Authors. Prometheus: Monitoring system and time series database. <https://prometheus.io>, 2026. CNCF graduated project; accessed July 2026.
- [27] Benjamin H. Sigelman, Luiz André Barroso, Mike Burrows, Pat Stephenson, Manoj Plakal, Donald Beaver, Saul Jaspán, and Chandan Shanbhag. Dapper, a large-scale distributed systems tracing infrastructure. Technical Report dapper-2010-1, Google, Inc., 2010.
- [28] Renan Souza, Timothy Poteet, Brian Etz, Daniel Rosendo, Amal Gueroudji, Woong Shin, Prasanna Balaprakash, and Rafael Ferreira da Silva. LLM agents for interactive workflow provenance: Reference architecture and evaluation methodology. In *SC '25 Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis (WORKS)*, 2025.
- [29] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. In *First Conference on Language Modeling (COLM)*, 2024.
- [30] Cui-Qing Yang and Barton P. Miller. Critical path analysis for the execution of parallel and distributed programs. In *Proceedings of the 8th International Conference on Distributed Computing Systems (ICDCS)*, pages 366–373, 1988.
- [31] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- [32] Andy B. Yoo, Morris A. Jette, and Mark Grondona. SLURM: Simple linux utility for resource management. In *Job Scheduling Strategies for Parallel Processing (JSSPP)*, volume 2862 of *Lecture Notes in Computer Science*, pages 44–60. Springer, 2003.